

combat/weapons/pistol.cpp

```
#include "pistol.h"
#include "hitscan-firing-mode.h"

namespace combat {
    namespace weapons {
        pistol::pistol(geometry::map_data const& map, engine::world const& world,
            audio::audio_mixer& mix, assets::asset_manager const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world), 8, 2.0f,
                ↪ 25.0f, 5,
                    mix, am, 1, 2, 1.0f) {}
    }
}
```

combat/weapons/plasma-gun.cpp

```
#include "plasma-gun.h"
#include "projectile-firing-mode.h"

namespace combat {
    namespace weapons {
        plasma_gun::plasma_gun(geometry::map_data& map, engine::world& world,
                                audio::audio_mixer& mix, assets::asset_manager const&
↪ am)
            : weapon(std::make_unique<projectile_firing_mode>(
                world, map, engine::faction::player, 30, 400.0f, 5.0f, 1.0f),
                    10, 1.0f, 200.0f, 3, mix, am, 11, 12, 5.0f) {})
        {}
    }
}
```

combat/weapons/rifle.cpp

```
#include "rifle.h"
#include "hitscan-firing-mode.h"

namespace combat {
    namespace weapons {
        rifle::rifle(geometry::map_data const& map, engine::world const& world,
                     audio::audio_mixer& mix, assets::asset_manager const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world), 20, 3.0f,
                    ↪ 35.0f, 4,
                      mix, am, 5, 6, 1.8f) {}
    }
}
```

combat/weapons/shotgun.cpp

```
#include "shotgun.h"
#include "hitscan-firing-mode.h"
#include "audio/audio-mixer.h"
#include "assets/asset-manager.h"

namespace combat {
    namespace weapons {
        shotgun::shotgun(geometry::map_data const& map, engine::world const& world,
                        audio::audio_mixer& mix, assets::asset_manager const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world), 8, 1.0f,
                ↪ 15.0f, 4,
                    mix, am, 7, 8, 2.0f) {}

        void shotgun::fire(math::vec2 pos, float angle) {
            if (!can_fire()) return;
            float step = spread / static_cast<float>(pellet_count - 1);
            float start_angle = angle - spread / 2.0f;
            for (int i = 0; i < pellet_count; ++i)
                ammo->spawn_bullet(pos, start_angle + static_cast<float>(i) * step,
                ↪ damage);
            --ammo_count;
            last_shot_time = 1.0f / fire_rate;
            if (fire_sound_id >= 0)
                ↪ mixer.play(assets.audio_clip_by_id(fire_sound_id));
        }
    }
}
```

combat/weapons/smg.cpp

```
#include "smg.h"
#include "hitscan-firing-mode.h"

namespace combat {
    namespace weapons {
        smg::smg(geometry::map_data const& map, engine::world const& world,
                  audio::audio_mixer& mix, assets::asset_manager const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world), 30, 10.0f,
                  ↵ 10.0f, 4,
                      mix, am, 3, 4, 1.5f) {}
    }
}
```

combat/weapons/sniper-rifle.cpp

```
#include "sniper-rifle.h"
#include "hitscan-firing-mode.h"

namespace combat {
    namespace weapons {
        sniper_rifle::sniper_rifle(geometry::map_data const& map, engine::world
↪ const& world,
                                   audio::audio_mixer& mix, assets::asset_manager
↪ const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world), 5, 0.5f,
↪ 100.0f, 3,
              mix, am, 9, 10, 2.5f) {}
    }
}
```

combat/weapons/weapon.cpp

```
#include "weapon.h"
#include "audio/audio-mixer.h"
#include "assets/asset-manager.h"
#include <algorithm>

namespace combat {
    namespace weapons {

        weapon::weapon(std::unique_ptr<firing_mode> firing, int max, float rate,
↪ float dmg, int reserve,
                        audio::audio_mixer& mix, assets::asset_manager const& am,
                        assets::audio_clip_id fire_snd, assets::audio_clip_id
↪ reload_snd,
                        float reload_dur)
            : ammo(std::move(firing)), ammo_count(max),
              max_ammo(max), reserve_mags(reserve), fire_rate(rate),
↪ last_shot_time(0.0f), damage(dmg),
              mixer(mix), assets(am), fire_sound_id(fire_snd),
↪ reload_sound_id(reload_snd),
              reload_duration(reload_dur), reload_timer(0.0f), reloading(false) {}

        bool weapon::can_fire() const {
            return ammo_count > 0 && last_shot_time <= 0.0f && !reloading;
        }

        void weapon::tick(float dt) {
            last_shot_time = std::max(0.0f, last_shot_time - dt);
            if (reloading) {
                reload_timer += dt;
                if (reload_timer >= reload_duration)
                    finish_reload();
            }
        }

        void weapon::resupply(int mags) {
            reserve_mags += mags;
        }

        void weapon::fire(math::vec2 pos, float angle) {
            if (!can_fire()) return;
            ammo->spawn_bullet(pos, angle, damage);
            --ammo_count;
            last_shot_time = 1.0f / fire_rate;
            if (fire_sound_id >= 0)
↪ mixer.play(assets.audio_clip_by_id(fire_sound_id));
        }

        void weapon::reload() {
            if (reloading) return;
            if (reserve_mags <= 0) return;
            if (ammo_count == max_ammo) return;
            reloading = true;
            reload_timer = 0.0f;
            if (reload_sound_id >= 0)
↪ mixer.play(assets.audio_clip_by_id(reload_sound_id));
        }
    }
}
```

```
void weapon::finish_reload() {  
    --reserve_mags;  
    ammo_count = max_ammo;  
    reloading = false;  
}  
}
```


combat/weapons/katana.cpp

```
#include "katana.h"
#include "hitscan-firing-mode.h"
#include "audio/audio-mixer.h"
#include "assets/asset-manager.h"

namespace combat {
    namespace weapons {
        katana::katana(geometry::map_data const& map, engine::world const& world,
                        audio::audio_mixer& mix, assets::asset_manager const& am)
            : weapon(std::make_unique<hitscan_firing_mode>(map, world, 60.0f, 32.0f),
                    ↪ 0, 1.5f, 50.0f, 0, mix, am, 13) {}

        bool katana::can_fire() const {
            return last_shot_time <= 0.0f;
        }

        void katana::fire(math::vec2 pos, float angle) {
            if (!can_fire()) return;
            if (ammo) ammo->spawn_bullet(pos, angle, damage);
            last_shot_time = 1.0f / fire_rate;
            if (fire_sound_id >= 0)
                ↪ mixer.play(assets.audio_clip_by_id(fire_sound_id));
        }

        void katana::reload() {}
    }
}
```

combat/weapons/hitscan-firing-mode.cpp

```
#include "hitscan-firing-mode.h"
#include "math/ray2.h"
#include "geometry/sidedef.h"
#include "engine/world.h"
#include <cmath>

namespace combat {
    namespace weapons {

        hitscan_firing_mode::hitscan_firing_mode(geometry::map_data const& m,
        ↪ engine::world const& w,
                                float range, float radius)
            : map(m), world_ref(w), max_range(range), hit_radius(radius) {}

        void hitscan_firing_mode::spawn_bullet(math::vec2 pos, float angle, float
        ↪ damage) {
            math::vec2 dir{-std::sin(angle), std::cos(angle)};
            math::ray2 ray{pos, dir};

            float wall_dist = max_range;

            for (auto const& e : map("linedefs"_f)) {
                geometry::linedef const& ld = e.value;
                if (ld.is_portal()) continue;
                math::vec2 hit;
                float dist = 0.0f, seg_len = 0.0f;
                if (ray.intersects(ld("seg"_f), hit, dist, seg_len) && dist <
                ↪ wall_dist)
                    wall_dist = dist;
            }

            engine::actor* hit_target = nullptr;
            float target_dist = wall_dist;

            for (auto [id, entity_ptr] : world_ref.get_entities()) {
                if (!entity_ptr) continue;
                auto* a = dynamic_cast<engine::actor*>(&*entity_ptr);
                if (!a || a->is_dead()) continue;
                // skip the shooter (distance 0)
                math::vec2 self_diff = (*a)("pos"_f) - pos;
                if (self_diff.sqr_len() < 1.0f) continue;

                math::vec2 to_actor = (*a)("pos"_f) - pos;
                float t = math::vec2::dot_product(to_actor, dir);
                if (t <= 0.0f || t > target_dist) continue;

                math::vec2 closest = pos + dir * t;
                math::vec2 offset = (*a)("pos"_f) - closest;
                if (offset.sqr_len() < hit_radius * hit_radius) {
                    target_dist = t;
                    hit_target = a;
                }
            }

            if (hit_target)
                hit_target->take_damage(damage);
        }
    }
}
```

}
}
}

combat/weapons/projectile-firing-mode.cpp

```
#include "projectile-firing-mode.h"
#include "engine/projectile.h"
#include "engine/world.h"
#include "geometry/map-data.h"
#include <cmath>

namespace combat {
    namespace weapons {

        projectile_firing_mode::projectile_firing_mode(engine::world& w,
        ↪ geometry::map_data& md,

                                                    engine::faction f,
                                                    assets::texture_id tex, float

        ↪ spd,

                                                    float lifetime, float scale)

            : world_ref(w), map_ref(md), team(f),
              projectile_tex(tex), projectile_speed(spd),
              projectile_lifetime(lifetime), projectile_scale(scale) {}

        void projectile_firing_mode::spawn_bullet(math::vec2 pos, float angle, float
        ↪ damage) {
            math::vec2 dir{-std::sin(angle), std::cos(angle)};

            math::vec2 spawn_pos = pos + dir * 20.0f;

            auto proj = std::make_unique<engine::projectile>(
                spawn_pos, 0.0f, projectile_tex, projectile_scale,
                dir, projectile_speed, damage, projectile_lifetime,
                team);

            auto* raw = &*proj;
            raw->world_ref = &world_ref;
            raw->map_ref = &map_ref;

            auto eid = world_ref.register_entity(std::move(proj));
            raw->self_id = eid;

            if (map_ref("root_node_id"_f) !=
            ↪ util::indexed_storage<geometry::bsp_node>::nullid) {
                auto sub_id = map_ref.get_subsector_id(spawn_pos);
                map_ref("subsectors"_f)[sub_id].add_sprite(raw);
            }
        }
    }
}
```

combat/burning.cpp

```
#include "burning.h"
#include "engine/actor.h"

namespace combat
{
    burning::burning(float const dur, unsigned int intens)
        : status_effect(dur, 0.5f, intens) {}

    void burning::affect(engine::actor& target)
    {
        float damage = static_cast<float>(intensity);

        target.take_true_damage(damage);
    }
}
```

combat/charmed.cpp

```
#include "charmed.h"
#include "engine/actor.h"

namespace combat
{
    charmed::charmed(float const dur, unsigned int intens)
        : status_effect(dur, 0.0f, intens) {}

    void charmed::on_apply(engine::actor& target)
    {
        target.team = engine::faction::player;
    }

    void charmed::on_expire(engine::actor& target)
    {
        target.team = engine::faction::enemy;
    }
}
```

combat/slowed.cpp

```
#include "slowed.h"
#include "engine/actor.h"
#include <algorithm>

namespace combat
{
    slowed::slowed(float const dur, unsigned int intens)
        : status_effect(dur, 0.0f, intens) {}

    void slowed::on_apply(engine::actor& target)
    {
        float slow_factor = std::clamp(1.0f - (intensity / 100.0f), 0.1f, 1.0f);
        target.movement_speed *= slow_factor;
    }

    void slowed::on_expire(engine::actor& target)
    {
        float slow_factor = std::clamp(1.0f - (intensity / 100.0f), 0.1f, 1.0f);
        target.movement_speed /= slow_factor;
    }
}
```

combat/status-effect.cpp

```
#include "status-effect.h"

namespace combat {

    status_effect::status_effect(float const dur, float tick_inter, unsigned int
↪ intens)
        : duration(dur), intensity(intens), tick_interval(tick_inter),
↪ tick_timer(0.0f) {}

    void status_effect::on_apply(engine::actor&) {}
    void status_effect::on_expire(engine::actor&) {}
    void status_effect::affect(engine::actor&) {}

    bool status_effect::is_expired() const { return duration <= 0.0f; }

    bool status_effect::tick(float dt) {
        duration -= dt;
        if (tick_interval <= 0.0f) return true; // non-ticking effect

        tick_timer += dt;
        if (tick_timer >= tick_interval) {
            tick_timer -= tick_interval;
            return false; // tick fires
        }
        return true; // between ticks - skip affect()
    }

}
```


engine/world.cpp

```
#include "world.h"

namespace engine {

    world::world() {};

    void world::update(float const dt) {
        for (auto const& [id, entity_ptr] : entities) {
            if (entity_ptr) {
                entity_ptr->update(dt);
            }
        }

        for (auto const id : deleted_entities) {
            entities.remove(id);
        }
        deleted_entities.clear();
    }

    entity& world::operator[] (util::indexed_storage<std::unique_ptr<entity>>::id_t
↪ const id) {
        return *entities[id];
    }

    entity const&
↪ world::operator[] (util::indexed_storage<std::unique_ptr<entity>>::id_t const id)
↪ const {
        return *entities[id];
    }

    util::indexed_storage<std::unique_ptr<entity>> const& world::get_entities() const
↪ {
        return entities;
    }

    util::indexed_storage<std::unique_ptr<entity>>::id_t
↪ world::register_entity(std::unique_ptr<entity> e) {
        return entities.add(std::move(e));
    }

    void world::delete_entity(util::indexed_storage<std::unique_ptr<entity>>::id_t
↪ const id) {
        deleted_entities.push_back(id);
    }

    entity* world::entity_from_id(util::indexed_storage<std::unique_ptr<entity>>::id_t
↪ const id) const {
        if (entities.contains(id)) {
            return entities[id].get();
        }
        return nullptr;
    }

}
```

engine/actor.cpp

```
#include "actor.h"

namespace engine {

    actor::actor(math::vec2 const p, float const z, assets::texture_id const tex,
    ↪ float const is, float const hp, float const shield, float const move_speed,
    ↪ faction const this_team)
        : renderable_entity(p, z, tex, is), health(hp, hp, shield, shield),
        ↪ movement_speed(move_speed), team(this_team) {}

    bool actor::is_dead() const { return health.is_dead(); }

    void actor::take_damage(float const dmg) {
        health.apply_damage(dmg);
        hit_flash = 0.15f;
    }
    void actor::take_true_damage(float const dmg) { health.apply_true_damage(dmg); }
    void actor::heal(float const amount) { health.apply_heal(amount); }
    void actor::add_shield(float const amount) { health.apply_shield(amount); }

    void actor::add_effect(std::unique_ptr<combat::status_effect> effect) {
        health.add_effect(std::move(effect), *this);
    }

    void actor::update(float dt) {
        health.process_effects(dt, *this);
        if (hit_flash > 0.0f) hit_flash -= dt;
    }

}
```

engine/projectile.cpp

```
#include "projectile.h"
#include "world.h"
#include "geometry/map-data.h"
#include "geometry/linedef.h"
#include "geometry/bsp-node.h"
#include "math/ray2.h"
#include <cmath>

namespace engine {

projectile::projectile(math::vec2 p, float z, assets::texture_id tex, float scale,
                      math::vec2 dir, float spd, float dmg, float life,
                      faction f, float radius)
: renderable_entity(p, z, tex, scale),
  direction(dir), speed(spd), damage(dmg), lifetime(life),
  hit_radius(radius), team(f) {}

void projectile::update(float dt) {
    if (lifetime <= 0.0f) return;

    bool has_bsp = map_ref &&
        (*map_ref)("root_node_id"_f) !=
        ↪ util::indexed_storage<geometry::bsp_node>::nullid;

    auto remove_and_delete = [&]() {
        if (has_bsp) {
            auto sub_id = map_ref->get_subsector_id(pos);
            (*map_ref)("subsectors"_f)[sub_id].remove_sprite(this);
        }
        lifetime = 0.0f;
        if (world_ref && self_id != 0)
            world_ref->delete_entity(self_id);
    };

    lifetime -= dt;
    if (lifetime <= 0.0f) {
        remove_and_delete();
        return;
    }

    math::vec2 new_pos = pos + direction * (speed * dt);

    // wall collision
    if (map_ref) {
        math::ray2 ray{pos, direction};
        float move_dist = speed * dt;

        for (auto const& e : (*map_ref)("linedefs"_f)) {
            geometry::linedef const& ld = e.value;
            if (ld.is_portal()) continue;
            math::vec2 hit;
            float dist = 0.0f, seg_len = 0.0f;
            if (ray.intersects(ld("seg"_f), hit, dist, seg_len) && dist > 0.0f && dist
                ↪ <= move_dist) {
                remove_and_delete();
                return;
            }
        }
    }
}
```

```

    }
}

// actor collision
if (world_ref) {
    for (auto [id, entity_ptr] : world_ref->get_entities()) {
        if (!entity_ptr) continue;
        auto* a = dynamic_cast<actor*>(&*entity_ptr);
        if (!a || a->is_dead()) continue;
        if (a->team == team) continue;

        math::vec2 diff = (*a)("pos"_f) - new_pos;
        if (diff.sqr_len() < hit_radius * hit_radius) {
            a->take_damage(damage);
            if (on_hit_effect) a->add_effect(on_hit_effect());
            remove_and_delete();
            return;
        }
    }
}

// move
if (map_ref)
    map_ref->move_to(this, new_pos);
else
    pos = new_pos;
}

}

```

geometry/sector.cpp

```
#include "geometry/sector.h"
#include <cstring>

namespace geometry {
    struct bin_sector {
        float floor_height, ceiling_height;
        std::uint32_t floor_tex, ceiling_tex;
        std::uint8_t light_level;
    } __attribute__((packed));

    sector::sector(float fh, float ch, assets::texture_id ft, assets::texture_id ct,
        ↪ std::uint8_t light) :
        floor_height(fh),
        ceiling_height(ch),
        floor_tex(ft),
        ceiling_tex(ct),
        light_level(light) {}

    std::vector<sector> sector::load_from_bin(util::resource const& res) {
        if (!res("beginning"_f) || res("size"_f) == 0) return {};

        size_t count = res("size"_f) / sizeof(bin_sector);
        auto const* data = reinterpret_cast<bin_sector const*>(res("beginning"_f));

        std::vector<sector> result;
        result.reserve(count);

        for (size_t i = 0; i < count; ++i) {
            result.emplace_back(data[i].floor_height, data[i].ceiling_height,
            ↪ data[i].floor_tex, data[i].ceiling_tex, data[i].light_level);
        }
        return result;
    }
}
```

geometry/bsp-node.cpp

```
#include "bsp-node.h"

namespace geometry {
    struct bin_bsp_node {
        float pl_x, pl_y, dir_x, dir_y;
        float f_top, f_bot, f_left, f_right;
        float b_top, b_bot, b_left, b_right;
        std::uint32_t front_child, back_child;
    } __attribute__((packed));

    bool bsp_node::is_pt_front_side(math::vec2 const& point) const {
        math::vec2 delta = point - pl_coord;
        return math::vec2::cross_product(pl_dir, delta) <= 0;
    }

    bsp_node::bsp_node(math::vec2 const pc, math::vec2 const pd, bounding_box const
    ↪ fb, bounding_box const bb, util::indexed_storage<bsp_node>::id_t f,
    ↪ util::indexed_storage<bsp_node>::id_t b) :
        pl_coord(pc),
        pl_dir(pd),
        front_box(fb),
        back_box(bb),
        front(f),
        back(b) {}

    std::vector<bsp_node> const bsp_node::load_from_bin(util::resource const& res) {
        std::vector<bsp_node> result;
        if (!res("beginning"_f) || res("size"_f) == 0) return result;

        size_t count = res("size"_f) / sizeof(bin_bsp_node);
        auto const* data = reinterpret_cast<bin_bsp_node const*>(res("beginning"_f));

        for (size_t i = 0; i < count; ++i) {
            bounding_box front_box = {
                float(data[i].f_top), float(data[i].f_bot),
                float(data[i].f_left), float(data[i].f_right)
            };
            bounding_box back_box = {
                float(data[i].b_top), float(data[i].b_bot),
                float(data[i].b_left), float(data[i].b_right)
            };

            math::vec2 pl_coord(float(data[i].pl_x), float(data[i].pl_y));
            math::vec2 pl_dir(float(data[i].dir_x), float(data[i].dir_y));

            // binary uses 0-based indices; indexed_storage IDs start at 1
            auto fix_child = [](std::uint32_t raw) -> std::uint32_t {
                if (raw & leaf_flag)
                    return leaf_flag | ((raw & ~leaf_flag) + 1);
                return raw + 1;
            };

            result.emplace_back(pl_coord, pl_dir, front_box, back_box,
            ↪ fix_child(data[i].front_child), fix_child(data[i].back_child));
        }
        return result;
    }
}
```

}
}

geometry/linedef.cpp

```
#include "geometry/linedef.h"

namespace geometry {
    struct bin_linedef {
        int v1_x, v1_y, v2_x, v2_y;
        std::uint32_t front, back;
    } __attribute__((packed));

    linedef::linedef(math::vec2 const p0, math::vec2 const p1,
↪ util::indexed_storage<sidedef>::id_t const f, util::indexed_storage<sidedef>::id_t
↪ const b) :
        seg(p0, p1),
        front(f),
        back(b) {}

    std::vector<linedef> linedef::load_from_bin(util::resource const& res) {
        std::vector<linedef> result;
        size_t count = res("size"_f) / sizeof(bin_linedef);
        auto const* data = reinterpret_cast<bin_linedef const*>(res("beginning"_f));

        for (size_t i = 0; i < count; ++i) {
            // binary uses 0-based indices; indexed_storage IDs start at 1
            auto front = data[i].front + 1;
            auto back = (data[i].back == 0xFFFFFFFF)
                ? util::indexed_storage<sidedef>::nullid
                : data[i].back + 1;
            result.emplace_back(math::vec2(float(data[i].v1_x), float(data[i].v1_y)),
↪ math::vec2(float(data[i].v2_x), float(data[i].v2_y)), front, back);
        }
        return result;
    }
}
```


geometry/map-data.cpp

```
#include "map-data.h"
#include <algorithm>

namespace geometry {

    util::indexed_storage<subsector>::id_t map_data::get_subsector_id(math::vec2
    ↪ const& pt) const {
        auto curr_id = root_node_id;

        while (true) {
            auto const& node = nodes[curr_id];

            if (node.is_pt_front_side(pt)) {
                if (node.is_front_leaf()) {
                    return node.get_front_index();
                }
                curr_id = node.get_front_index();
            } else {
                if (node.is_back_leaf()) {
                    return node.get_back_index();
                }
                curr_id = node.get_back_index();
            }
        }
    }

    void map_data::move_to(rendering::sprite* spr, math::vec2 const& new_pos) {
        if (!spr) return;

        auto old_sub_id = get_subsector_id((*spr)("pos"_f));
        auto new_sub_id = get_subsector_id(new_pos);

        (*spr)("pos"_f) = new_pos;

        if (old_sub_id != new_sub_id) {
            subsectors[old_sub_id].remove_sprite(spr);
            subsectors[new_sub_id].add_sprite(spr);
        }
    }

    map_data map_data::load_from_bin(util::resource const& sectors_res, util::resource
    ↪ const& sidedefs_res, util::resource const& linedefs_res, util::resource const&
    ↪ subsectors_res, util::resource const& nodes_res, util::resource const&
    ↪ monsters_res, util::resource const& pickups_res) {
        map_data map;

        map.sectors =
    ↪ util::indexed_storage<sector>(sector::load_from_bin(sectors_res));
        map.sidedefs =
    ↪ util::indexed_storage<sidedef>(sidedef::load_from_bin(sidedefs_res));
        map.linedefs =
    ↪ util::indexed_storage<linedef>(linedef::load_from_bin(linedefs_res));
        map.subsectors =
    ↪ util::indexed_storage<subsector>(subsector::load_from_bin(subsectors_res));
    }
```

```

    auto loaded_nodes =
    ↪ util::indexed_storage<bsp_node>(bsp_node::load_from_bin(nodes_res));

    if (loaded_nodes.size() != 0) {
        // IDs in indexed_storage start at 1; the last node (root) has ID =
        ↪ size()
        map.root_node_id = loaded_nodes.size();
    } else {
        map.root_node_id = util::indexed_storage<bsp_node>::nullid;
    }

    map.nodes = std::move(loaded_nodes);

    map.monster_spawns = monster_spawn::load_from_bin(monsters_res);
    map.pickup_spawns = pickup_spawn::load_from_bin(pickups_res);

    return map;
}
}

```

geometry/monster-spawn.cpp

```
#include "geometry/monster-spawn.h"

namespace geometry {
    monster_spawn::monster_spawn(std::uint32_t t, math::vec2 p, float height)
        : type(t), pos(p), z(height) {}

    class bin_monster_spawn {
    public:
        std::uint32_t type;
        int x, y;
        float z;
    } __attribute__((packed));

    std::vector<monster_spawn> monster_spawn::load_from_bin(util::resource const& res)
    ↪ {
        if (!res("beginning"_f) || res("size"_f) == 0) return {};

        std::vector<monster_spawn> result;
        size_t count = res("size"_f) / sizeof(bin_monster_spawn);
        auto const* data = reinterpret_cast<bin_monster_spawn
        ↪ const*>(res("beginning"_f));

        for (size_t i = 0; i < count; ++i) {
            result.emplace_back(data[i].type, math::vec2(float(data[i].x),
            ↪ float(data[i].y)), data[i].z);
        }
        return result;
    }
}
```

geometry/pickup-spawn.cpp

```
#include "geometry/pickup-spawn.h"

namespace geometry {
    pickup_spawn::pickup_spawn(std::uint32_t t, std::uint32_t st, math::vec2 p, float
↪ height)
        : type(t), subtype(st), pos(p), z(height) {}

    class bin_pickup_spawn {
    public:
        std::uint32_t type;
        std::uint32_t subtype;
        int x, y;
        float z;
    } __attribute__((packed));

    std::vector<pickup_spawn> pickup_spawn::load_from_bin(util::resource const& res) {
        if (!res("beginning"_f) || res("size"_f) == 0) return {};

        std::vector<pickup_spawn> result;
        size_t count = res("size"_f) / sizeof(bin_pickup_spawn);
        auto const* data = reinterpret_cast<bin_pickup_spawn
↪ const*>(res("beginning"_f));

        for (size_t i = 0; i < count; ++i) {
            result.emplace_back(data[i].type, data[i].subtype,
↪ math::vec2(float(data[i].x), float(data[i].y)), data[i].z);
        }
        return result;
    }
}
```

geometry/sidedef.cpp

```
#include "geometry/sidedef.h"

namespace geometry {
    struct bin_sidedef {
        std::uint32_t facing_sector;
        std::uint32_t upper_tex, middle_tex, lower_tex;
    } __attribute__((packed));

    sidedef::sidedef(util::indexed_storage<sector>::id_t sector, assets::texture_id
↪ ut, assets::texture_id mt, assets::texture_id lt) :
        facing_sector(sector),
        upper_tex(ut),
        middle_tex(mt),
        lower_tex(lt) {}

    std::vector<sidedef> sidedef::load_from_bin(util::resource const& res) {
        size_t count = res("size"_f) / sizeof(bin_sidedef);
        auto const* data = reinterpret_cast<bin_sidedef const*>(res("beginning"_f));

        std::vector<sidedef> result;
        result.reserve(count);

        for (size_t i = 0; i < count; ++i) {
            // binary uses 0-based indices; indexed_storage IDs start at 1
            result.emplace_back(data[i].facing_sector + 1, data[i].upper_tex,
↪ data[i].middle_tex, data[i].lower_tex);
        }
        return result;
    }
}
```

geometry/subsector.cpp

```
#include "geometry/subsector.h"

#include <utility>
#include <algorithm>

namespace geometry {
    struct bin_subsector {
        std::uint32_t line_count;
        std::uint32_t first_line_id;
    } __attribute__((packed));

    subsector::subsector(std::vector<util::indexed_storage<linedef>::id_t> l) :
        lines(std::move(l)) {}

    std::vector<subsector> subsector::load_from_bin(util::resource const& res) {
        size_t count = res("size"_f) / sizeof(bin_subsector);
        auto const* data = reinterpret_cast<bin_subsector const*>(res("beginning"_f));

        std::vector<subsector> result;
        result.reserve(count);

        for (size_t i = 0; i < count; ++i) {
            std::vector<util::indexed_storage<linedef>::id_t> lines;
            lines.reserve(data[i].line_count);
            for (std::uint32_t j = 0; j < data[i].line_count; ++j) {
                // binary uses 0-based indices; indexed_storage IDs start at 1
                lines.push_back(data[i].first_line_id + j + 1);
            }
            result.emplace_back(std::move(lines));
        }
        return result;
    }

    void subsector::remove_sprite(rendering::sprite* spr) {
        auto it = std::find(sprites.begin(), sprites.end(), spr);
        if (it != sprites.end())
            sprites.erase(it);
    }

    void subsector::add_sprite(rendering::sprite* spr) {
        sprites.push_back(spr);
    }
}
```

input/input-backend.cpp

```
#include "input-backend.h"

namespace input {
    input_backend::input_backend() : bad(false) {}

    bool input_backend::is_bad() const { return bad; }

    key input_backend::to_key(char c) {
        if (c >= 'a' && c <= 'z') return static_cast<key>(static_cast<int>(key::a) +
            ↪ (c - 'a'));
        if (c >= 'A' && c <= 'Z') return static_cast<key>(static_cast<int>(key::a) +
            ↪ (c - 'A'));
        if (c >= '0' && c <= '9') return static_cast<key>(static_cast<int>(key::n0) +
            ↪ (c - '0'));

        switch (c) {
            case ' ': return key::space;
            case '\t': return key::tab;
            case '\n': return key::enter;
            case '`': return key::backtick;
            case '-': return key::hyphen;
            case '=': return key::equals;
            case '[': return key::open_bracket;
            case ']': return key::close_bracket;
            case '\\': return key::backslash;
            case ';': return key::semicolon;
            case '\': return key::quote;
            case ',': return key::comma;
            case '.': return key::period;
            case '/': return key::slash;
            default: return key::unknown;
        }
    }
}
```

input/evdev/backend.cpp

```
#include "backend.h"
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <string>
#include <sys/epoll.h>

namespace input {
    namespace evdev {
        backend::backend() :
            mouse({0, 0, false, false, false}),
            last_abs_x(-1),
            last_abs_y(-1) {
            probe_devices();
            if (devices.empty()) bad = true;
        }

        backend::~backend() {
            for (auto& dev : devices) {
                ioctl(dev.fd, EVIOCGRAB, 0);
                close(dev.fd);
            }
        }

        void backend::probe_devices() {
            for (int i = 0; i < 32; ++i) {
                std::string path = "/dev/input/event" + std::to_string(i);
                probe_device(path);
            }
            // for (int i = 0; i < 32; ++i) {
            //     std::string path = "/dev/input/mouse" + std::to_string(i);
            //     probe_device(path);
            // }
            // probe_device("/dev/input/mice");

            epoll_fd = epoll_create1(0);
            for (auto& dev : devices) {
                struct epoll_event ev{};
                ev.events = EPOLLIN;
                ev.data.fd = dev.fd;
                epoll_ctl(epoll_fd, EPOLL_CTL_ADD, dev.fd, &ev);
            }
        }

        void backend::probe_device(std::string const& dev_name) {
            int fd = open(dev_name.c_str(), O_RDONLY | O_NONBLOCK);
            if (fd < 0) return;

            unsigned long key_bitmask[(KEY_CNT + (sizeof(long) * 8) - 1) /
                (sizeof(long) * 8)] = {0};
            unsigned long rel_bitmask[(REL_CNT + (sizeof(long) * 8) - 1) /
                (sizeof(long) * 8)] = {0};
            unsigned long abs_bitmask[(ABS_CNT + (sizeof(long) * 8) - 1) /
                (sizeof(long) * 8)] = {0};
        }
    }
}
```



```

bool has_keys = ioctl(fd, EVIOCGBIT(EV_KEY, sizeof(key_bitmask)),
↳ key_bitmask) >= 0;
bool has_rel = ioctl(fd, EVIOCGBIT(EV_REL, sizeof(rel_bitmask)),
↳ rel_bitmask) >= 0;
bool has_abs = ioctl(fd, EVIOCGBIT(EV_ABS, sizeof(abs_bitmask)),
↳ abs_bitmask) >= 0;

if (has_keys || has_rel || has_abs) {
    bool is_kb = has_keys && (key_bitmask[KEY_ENTER / (sizeof(long) * 8)]
↳ & (1UL << (KEY_ENTER % (sizeof(long) * 8))));

    bool has_left_click = has_keys && (key_bitmask[BTN_LEFT /
↳ (sizeof(long) * 8)] & (1UL << (BTN_LEFT % (sizeof(long) * 8))));
    bool has_x_axis = has_rel && (rel_bitmask[REL_X / (sizeof(long) * 8)]
↳ & (1UL << (REL_X % (sizeof(long) * 8))));
    bool has_abs_x = has_abs && (abs_bitmask[ABS_X / (sizeof(long) * 8)] &
↳ (1UL << (ABS_X % (sizeof(long) * 8))));

    bool is_ms = has_left_click || has_x_axis || has_abs_x;

    if (is_kb || is_ms) {

        struct input_event discard;
        while (read(fd, &discard, sizeof(discard)) > 0);

        ioctl(fd, EVIOCGRAB, 1);
        devices.push_back({fd, is_kb, is_ms});
        return;
    }
}
close(fd);
}

```

```

uint16_t backend::map_to_native(key k) const {
    switch (k) {

        case key::a: return KEY_A; case key::b: return KEY_B; case key::c:
↳ return KEY_C;
        case key::d: return KEY_D; case key::e: return KEY_E; case key::f:
↳ return KEY_F;
        case key::g: return KEY_G; case key::h: return KEY_H; case key::i:
↳ return KEY_I;
        case key::j: return KEY_J; case key::k: return KEY_K; case key::l:
↳ return KEY_L;
        case key::m: return KEY_M; case key::n: return KEY_N; case key::o:
↳ return KEY_O;
        case key::p: return KEY_P; case key::q: return KEY_Q; case key::r:
↳ return KEY_R;
        case key::s: return KEY_S; case key::t: return KEY_T; case key::u:
↳ return KEY_U;
        case key::v: return KEY_V; case key::w: return KEY_W; case key::x:
↳ return KEY_X;
        case key::y: return KEY_Y; case key::z: return KEY_Z;

        case key::n0: return KEY_0; case key::n1: return KEY_1; case key::n2:
↳ return KEY_2;
        case key::n3: return KEY_3; case key::n4: return KEY_4; case key::n5:
↳ return KEY_5;
    }
}

```

```

case key::n6: return KEY_6; case key::n7: return KEY_7; case key::n8:
↳ return KEY_8;
case key::n9: return KEY_9;

case key::f1: return KEY_F1; case key::f2: return KEY_F2; case
↳ key::f3: return KEY_F3;
case key::f4: return KEY_F4; case key::f5: return KEY_F5; case
↳ key::f6: return KEY_F6;
case key::f7: return KEY_F7; case key::f8: return KEY_F8; case
↳ key::f9: return KEY_F9;
case key::f10: return KEY_F10; case key::f11: return KEY_F11; case
↳ key::f12: return KEY_F12;

case key::esc: return KEY_ESC; case key::tab: return KEY_TAB; case
↳ key::caps_lock: return KEY_CAPSLOCK;
case key::l_shift: return KEY_LEFTSHIFT; case key::l_ctrl: return
↳ KEY_LEFTCTRL;
case key::l_meta: return KEY_LEFTMETA; case key::l_alt: return
↳ KEY_LEFTALT;
case key::space: return KEY_SPACE; case key::r_alt: return
↳ KEY_RIGHTALT;
case key::r_meta: return KEY_RIGHTMETA; case key::r_ctrl: return
↳ KEY_RIGHTCTRL;
case key::r_shift: return KEY_RIGHTSHIFT; case key::enter: return
↳ KEY_ENTER;
case key::backspace: return KEY_BACKSPACE;

case key::print_screen: return KEY_PRINT; case key::scroll_lock:
↳ return KEY_SCROLLLOCK;
case key::pause: return KEY_PAUSE; case key::insert: return
↳ KEY_INSERT;
case key::home: return KEY_HOME; case key::page_up: return
↳ KEY_PAGEUP;
case key::del: return KEY_DELETE; case key::end: return KEY_END;
case key::page_down: return KEY_PAGEDOWN;
case key::up: return KEY_UP; case key::down: return
↳ KEY_DOWN;
case key::left: return KEY_LEFT; case key::right: return
↳ KEY_RIGHT;

case key::backtick: return KEY_GRAVE; case key::hyphen: return
↳ KEY_MINUS;
case key::equals: return KEY_EQUAL; case key::open_bracket: return
↳ KEY_LEFTBRACE;
case key::close_bracket: return KEY_RIGHTBRACE; case key::backslash:
↳ return KEY_BACKSLASH;
case key::semicolon: return KEY_SEMICOLON; case key::quote: return
↳ KEY_APOSTROPHE;
case key::comma: return KEY_COMMA; case key::period: return
↳ KEY_DOT;
case key::slash: return KEY_SLASH;

case key::num_lock: return KEY_NUMLOCK; case key::kp_slash: return
↳ KEY_KP_SLASH;
case key::kp_asterisk: return KEY_KPASTERISK; case key::kp_minus:
↳ return KEY_KPMINUS;
case key::kp_plus: return KEY_KPPLUS; case key::kp_enter: return
↳ KEY_KPENTER;
case key::kp_dot: return KEY_KPDOT;

```

```

        case key::kp0: return KEY_KP0; case key::kp1: return KEY_KP1; case
        ↪ key::kp2: return KEY_KP2;
        case key::kp3: return KEY_KP3; case key::kp4: return KEY_KP4; case
        ↪ key::kp5: return KEY_KP5;
        case key::kp6: return KEY_KP6; case key::kp7: return KEY_KP7; case
        ↪ key::kp8: return KEY_KP8;
        case key::kp9: return KEY_KP9;
        default: return 0;
    }
}

void backend::update() {
    struct epoll_event events[32];
    int n = epoll_wait(epoll_fd, events, 32, 0);

    for (int i = 0; i < n; ++i) {

        struct input_event ev_buffer[64];

        ssize_t bytes_read;
        while ((bytes_read = read(events[i].data.fd, ev_buffer,
        ↪ sizeof(ev_buffer))) > 0) {
            int num_events = bytes_read / sizeof(struct input_event);
            for (int j = 0; j < num_events; ++j) {
                process_event(ev_buffer[j]);
            }
        }
    }
}

void backend::process_event(struct input_event const& ev) {
    if (ev.type == EV_KEY && ev.code < KEY_CNT) {
        key_map[ev.code] = (ev.value > 0);
        if (ev.code == BTN_LEFT) mouse.left = (ev.value > 0);
        if (ev.code == BTN_RIGHT) mouse.right = (ev.value > 0);
        if (ev.code == BTN_MIDDLE) mouse.middle = (ev.value > 0);

        if (ev.code == BTN_TOUCH && ev.value == 0) {
            last_abs_x = -1;
            last_abs_y = -1;
        }
    } else if (ev.type == EV_REL) {
        if (ev.code == REL_X) mouse.x += ev.value;
        if (ev.code == REL_Y) mouse.y += ev.value;
    } else if (ev.type == EV_ABS) {
        if (ev.code == ABS_X) {
            if (last_abs_x != -1) mouse.x += (ev.value - last_abs_x);
            last_abs_x = ev.value;
        }
        if (ev.code == ABS_Y) {
            if (last_abs_y != -1) mouse.y += (ev.value - last_abs_y);
            last_abs_y = ev.value;
        }
    }
}

bool backend::is_key_down(key k) const {
    uint16_t native = map_to_native(k);
    return native != 0 && key_map[native];
}

```

```
}

mouse_state backend::get_mouse_state() const { return mouse; }

void backend::reset_mouse_state(int nx, int ny) {
    mouse.x = nx;
    mouse.y = ny;
    mouse.left = mouse.right = mouse.middle = false;
}

}
```

math/segment.cpp

```
#include "segment.h"

float math::segment::angle() const {
    return (point1 - point0).angle();
}

float math::segment::len() const {
    return (point1 - point0).len();
}

float math::segment::sqr_len() const {
    return (point1 - point0).sqr_len();
}
```

math/vec3.cpp

```
#include "vec3.h"

#include <cmath>

constexpr float math::vec3::sqr_len() const {
    return dot_product(*this, *this);
}

constexpr float math::vec3::len() const {
    return std::sqrt(sqr_len());
}

constexpr math::vec3 const math::vec3::normalized() const {
    float length = len();
    if (!length) {
        return vec3(0.0f, 0.0f, 0.0f);
    }
    return *this / length;
}
```

math/vec2.cpp

```
#include "vec2.h"
#include "vec3.h"

#include <cmath>

math::vec2::vec2(vec3 const& vec) : x(vec("x"_f)), y(vec("y"_f)) {}

float
math::vec2::dot_product(vec2 const first, vec2 const second) {
    return first.x * second.x + first.y * second.y;
}

float
math::vec2::cross_product(vec2 const first, vec2 const second) {
    return first.x * second.y - first.y * second.x;
}

float
math::vec2::sqr_len() const {
    return dot_product(*this, *this);
}

float
math::vec2::len() const {
    return std::sqrt(x*x+y*y);
}

math::vec2 const
math::vec2::normalized() const {
    float length = len();
    if (!length) {
        return vec2(float(0.0f), float(0.0f));
    }
    return *this / length;
}

math::vec2 const
math::vec2::perpendicular() const {
    return vec2(-y, x);
}

float
math::vec2::angle() const {
    return std::atan2(y, x);
}

float
math::vec2::angle_between(vec2 const first, vec2 const second) {
    float dot = dot_product(first, second);
    float len1 = first.len();
    float len2 = second.len();

    if (!len1 || !len2) {
        return float(0.0f);
    }
}
```

```

    float cos_theta = dot / (len1 * len2);

    if (cos_theta > float(1.0f)) cos_theta = float(1.0f);
    if (cos_theta < float(-1.0f)) cos_theta = float(-1.0f);

    return std::acos(cos_theta);
}

math::vec2 const
math::vec2::rotate(vec2 const vec, float const angle) {
    float c = std::cos(angle);
    float s = std::sin(angle);

    return vec2(
        vec.x * c - vec.y * s,
        vec.x * s + vec.y * c
    );
}

math::vec2 const
math::vec2::rotate_with_known_trig(vec2 const vec, float const cos, float const sin) {
    return vec2(
        vec.x * cos - vec.y * sin,
        vec.x * sin + vec.y * cos
    );
}

```


math/ray2.cpp

```
#include "ray2.h"

math::ray2 const math::ray2::rotate(ray2 const ray, float const angle) {
    return ray2(ray.end, vec2::rotate(ray.direction, angle));
}

bool math::ray2::intersects(segment const seg, vec2 &hit_point, float &distance, float
↪ &seg_len) const {
    vec2 p = end;
    vec2 r = direction;
    vec2 q = seg("point0"_f);
    vec2 s = seg("point1"_f) - seg("point0"_f);

    float r_cross_s = math::vec2::cross_product(r, s);
    float q_minus_p_cross_r = math::vec2::cross_product(q - p, r);

    if (!r_cross_s) {
        return false;
    }

    float t = math::vec2::cross_product(q - p, s) / r_cross_s;
    float u = q_minus_p_cross_r / r_cross_s;

    if (t >= 0.0f && u >= 0.0f && u <= 1.0f) {
        hit_point = p + r * t;
        distance = t * r.len();
        seg_len = u * s.len();
        return true;
    }

    return false;
}
```

rendering/drm-kms/crtc.cpp

```
#include "crtc.h"
#include <drm/drm.h>

namespace rendering {
    namespace drm_kms {
        crtc::crtc(device_context const& d, uint32_t id)
            : dev(d), crtc_id(id) {}

        bool crtc::set_config(std::uint32_t fb_id, uint32_t conn_id, mode const& m) {
            struct drm_mode_crtc crtc_req = m.create_crtc_req(crtc_id, fb_id,
                ↪ &conn_id);
            return dev.ioctl(DRM_IOCTL_MODE_SETCRTC, &crtc_req) == 0;
        }

        bool crtc::page_flip(uint32_t fb_id) const {
            struct drm_mode_crtc_page_flip flip_req = {};
            flip_req.crtc_id = crtc_id;
            flip_req.fb_id = fb_id;
            return dev.ioctl(DRM_IOCTL_MODE_PAGE_FLIP, &flip_req) == 0;
        }
    }
}
```

rendering/drm-kms/device-context.cpp

```
#include "device-context.h"
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <drm/drm.h>
#include <sys/mman.h>

namespace rendering {
    namespace drm_kms {
        device_context::device_context(std::string const& path) {
            fd = open(path.c_str(), O_RDWR | O_CLOEXEC);

            if (is_valid()) {
                ioctl(DRM_IOCTL_SET_MASTER, nullptr);
            }
        }

        device_context::~device_context() {
            if (is_valid()) {
                ioctl(DRM_IOCTL_DROP_MASTER, nullptr);
                close(fd);
            }
        }

        bool device_context::is_valid() const {
            return fd >= 0;
        }

        int device_context::ioctl(unsigned long request, void* arg) const {
            return ::ioctl(fd, request, arg);
        }

        void *device_context::mmap(void *addr, size_t len, int prot, int flags, off_t
↵ off) const {
            void *memory = ::mmap(addr, len, prot, flags, fd, off);

            if(memory == MAP_FAILED) {
                memory = nullptr;
            }

            return memory;
        }
    }
}
```

rendering/drm-kms/framebuffer.cpp

```
#include "framebuffer.h"
#include "crtc.h"
#include <drm/drm.h>
#include <cstring>
#include <sys/mman.h>

namespace rendering {
    namespace drm_kms {
        framebuffer::framebuffer(device_context const& d, uint32_t width, uint32_t
↪ height, bool* const success)
            : dev(d), handle(0), fb_id(0), mmio_ptr(nullptr), size(0), pitch(0) {

            struct drm_mode_create_dumb create_req = {};
            create_req.width = width;
            create_req.height = height;
            create_req.bpp = 32;

            if (dev.ioctl(DRM_IOCTL_MODE_CREATE_DUMB, &create_req) < 0) {
                if (success) *success = false;
                return;
            }

            handle = create_req.handle;
            pitch = create_req.pitch;
            size = create_req.size;

            struct drm_mode_fb_cmd fb_req = {};
            fb_req.width = width;
            fb_req.height = height;
            fb_req.pitch = pitch;
            fb_req.bpp = 32;
            fb_req.depth = 24;
            fb_req.handle = handle;

            if (dev.ioctl(DRM_IOCTL_MODE_ADDFB, &fb_req) < 0) {
                if (success) *success = false;
                return;
            }
            fb_id = fb_req.fb_id;

            struct drm_mode_map_dumb map_req = {};
            map_req.handle = handle;
            if (dev.ioctl(DRM_IOCTL_MODE_MAP_DUMB, &map_req) < 0) {
                if (success) *success = false;
                return;
            }

            mmio_ptr = static_cast<uint32_t*>(dev.mmap(nullptr, size, PROT_READ |
↪ PROT_WRITE, MAP_SHARED, map_req.offset));
            if (mmio_ptr == nullptr) {
                if (success) *success = false;
                return;
            }

            if (success) *success = true;
        }
    }
}
```

```

framebuffer::~framebuffer() {
    if (mmio_ptr != nullptr) {
        munmap(mmio_ptr, size);
    }
    if (fb_id != 0) {
        dev.ioctl(DRM_IOCTL_MODE_RMFB, &fb_id);
    }
    if (handle != 0) {
        struct drm_mode_destroy_dumb destroy_req = {};
        destroy_req.handle = handle;
        dev.ioctl(DRM_IOCTL_MODE_DESTROY_DUMB, &destroy_req);
    }
}

void framebuffer::copy_from(uint32_t const* src, size_t pixel_count) const {
    std::memcpy(mmio_ptr, src, pixel_count * sizeof(uint32_t));
}

bool framebuffer::flip_onto(crtc const& c) const {
    return c.page_flip(fb_id);
}

bool framebuffer::apply_config(crtc &c, std::uint32_t const connector_id, mode
↪ const& m) const {
    return c.set_config(fb_id, connector_id, m);
}
}
}

```

rendering/drm-kms/mode.cpp

```
#include "mode.h"

namespace rendering {
    namespace drm_kms {
        mode::mode(struct drm_mode_modeinfo const& info)
            : kernel_mode(info) {}

        struct drm_mode_crtc mode::create_crtc_req(std::uint32_t const crtc_id,
        ↪ std::uint32_t const fb_id, std::uint32_t const *const conn_id) const {
            struct drm_mode_crtc crtc_req = {};

            crtc_req.crtc_id = crtc_id;
            crtc_req.fb_id = fb_id;
            crtc_req.set_connectors_ptr = reinterpret_cast<std::uint64_t>(conn_id);
            crtc_req.count_connectors = 1;
            crtc_req.mode = kernel_mode;
            crtc_req.mode_valid = 1;

            return crtc_req;
        }

        unsigned int mode::operator()(util::component_tag<"x_res">) const {
            return kernel_mode.hdisplay;
        }

        unsigned int mode::operator()(util::component_tag<"y_res">) const {
            return kernel_mode.vdisplay;
        }

        unsigned int mode::operator()(util::component_tag<"refresh_hz">) const {
            return kernel_mode.vrefresh;
        }

        bool mode::operator()(util::component_tag<"has_vsync">) const {
            return kernel_mode.flags & (DRM_MODE_FLAG_PVSYNCR | DRM_MODE_FLAG_NVSYNCR);
        }
    }
}
```

rendering/drm-kms/backend.cpp

```
#include "backend.h"
#include <drm/drm.h>
#include <string>
#include <cstring>
#include "mode.h"

namespace rendering {
    namespace drm_kms {
        backend::backend() : is_bad(false), front_buffer_index(0),
↪ has_original_state(false) {
            bool pipeline_found = false;

            for (int i = 0; i < 64; ++i) {
                std::string device_path = "/dev/dri/card" + std::to_string(i);
                dev = std::make_unique<device_context>(device_path);

                if (!dev->is_valid()) {
                    continue;
                }

                struct drm_mode_card_res res = {};
                if (dev->iocctl(DRM_IOCTL_MODE_GETRESOURCES, &res) < 0) {
                    continue;
                }

                std::vector<uint32_t> connectors(res.count_connectors);
                std::vector<uint32_t> encoders(res.count_encoders);
                std::vector<uint32_t> crtcs(res.count_crtcs);

                res.connector_id_ptr = reinterpret_cast<uint64_t>(connectors.data());
                res.encoder_id_ptr = reinterpret_cast<uint64_t>(encoders.data());
                res.crtc_id_ptr = reinterpret_cast<uint64_t>(crtcs.data());

                if (dev->iocctl(DRM_IOCTL_MODE_GETRESOURCES, &res) < 0) {
                    continue;
                }

                for (uint32_t conn_id : connectors) {
                    struct drm_mode_get_connector conn_req = {};
                    conn_req.connector_id = conn_id;
                    dev->iocctl(DRM_IOCTL_MODE_GETCONNECTOR, &conn_req);

                    if (conn_req.connection == 1 && conn_req.encoder_id != 0) {
                        active_connector = std::make_unique<connector>(*dev, conn_id);

                        struct drm_mode_get_encoder enc_req = {};
                        enc_req.encoder_id = conn_req.encoder_id;
                        dev->iocctl(DRM_IOCTL_MODE_GETENCODER, &enc_req);

                        if (enc_req.crtc_id != 0) {
                            active_crtc = std::make_unique<crtc>(*dev,
↪ enc_req.crtc_id);

                            struct drm_mode_crtc old_crtc_req = {};
                            old_crtc_req.crtc_id = enc_req.crtc_id;
```

```

        if (dev->iocctl(DRM_IOCTL_MODE_GETCRTC, &old_crtc_req) ==
↪ 0) {
            original_fb_id      = old_crtc_req.fb_id;
            original_connector_id = conn_id;
            original_mode        = old_crtc_req.mode;
            has_original_state   = true;
        }

        pipeline_found = true;
        break;
    }
}

    if (pipeline_found) break;
}

    if (!pipeline_found) {
        is_bad = true;
    }
}

backend::~~backend() {
    if (has_original_state && active_crtc && dev && dev->is_valid()) {
        mode_restore_mode(original_mode);
        active_crtc->set_config(original_fb_id, original_connector_id,
↪ restore_mode);
    }
}

std::vector<std::unique_ptr<rendering_mode const>>
↪ backend::operator()(util::component_tag<"modes">) {
    if (bad()) return {};

    auto drm_modes = active_connector->probe_modes();
    std::vector<std::unique_ptr<rendering_mode const>> modes;
    for (auto& m : drm_modes) {
        modes.push_back(std::move(m));
    }
    return modes;
}

void backend::push_mode(std::unique_ptr<rendering_mode const> pushed_mode) {
    if (bad() || !pushed_mode) return;

    auto* drm_mode = static_cast<mode const*>(pushed_mode.get());

    buffers[0] = std::make_unique<framebuffer>(*dev, (*drm_mode)("x_res"_f),
↪ (*drm_mode)("y_res"_f));
    buffers[1] = std::make_unique<framebuffer>(*dev, (*drm_mode)("x_res"_f),
↪ (*drm_mode)("y_res"_f));

    front_buffer_index = 0;

    if (!active_connector->apply_config(*active_crtc,
↪ *buffers[front_buffer_index], *drm_mode)) {
        is_bad = true;
        return;
    }
}

```



```

        current_mode.reset(static_cast<mode const*>(pushed_mode.release()));

        shadow.resize(buffers[0]->size / sizeof(uint32_t));
    }

    bool backend::bad() const {
        return is_bad || !dev->is_valid();
    }

    unsigned int backend::operator()(util::component_tag<"width">) {
        return current_mode ? (*current_mode.get())("x_res"_f) : 0;
    }

    unsigned int backend::operator()(util::component_tag<"height">) {
        return current_mode ? (*current_mode.get())("y_res"_f) : 0;
    }

    unsigned int backend::operator()(util::component_tag<"pitch">) {
        if (!buffers[0] || !buffers[1]) return 0;
        int back_index = 1 - front_buffer_index;
        return buffers[back_index]->pitch;
    }

    std::uint32_t* backend::operator()(util::component_tag<"mmio">) {
        if (shadow.empty()) return nullptr;
        return shadow.data();
    }

    void backend::wait_for_vsync() {
        if (bad() || !active_crtc) return;

        union drm_wait_vblank vbl = {};
        vbl.request.type = (drm_vblank_seq_type)(_DRM_VBLANK_RELATIVE);
        vbl.request.sequence = 1;

        dev->iocctl(DRM_IOCTL_WAIT_VBLANK, &vbl);
    }

    void backend::flush() {
        if (bad() || !active_crtc || !buffers[0] || !buffers[1]) return;

        int back_index = 1 - front_buffer_index;

        buffers[back_index]->copy_from(shadow.data(), shadow.size());

        if (buffers[back_index]->flip_onto(*active_crtc)) {
            front_buffer_index = back_index;
        }
    }
}

```

rendering/drm-kms/connector.cpp

```
#include "connector.h"
#include "crtc.h"
#include "framebuffer.h"
#include <drm/drm.h>
#include <vector>

namespace rendering {
    namespace drm_kms {
        connector::connector(device_context const& d, uint32_t id, bool* const
↪ success)
            : dev(d), connector_id(id), encoder_id(0) {

            struct drm_mode_get_connector conn_req = {};
            conn_req.connector_id = connector_id;

            if (dev.ioctl(DRM_IOCTL_MODE_GETCONNECTOR, &conn_req) < 0) {
                if (success) *success = false;
                return;
            }

            if (success) *success = true;
            encoder_id = conn_req.encoder_id;
        }

        std::vector<std::unique_ptr<mode const>> connector::probe_modes() {
            std::vector<std::unique_ptr<mode const>> modes;

            struct drm_mode_get_connector conn_req = {};
            conn_req.connector_id = connector_id;

            if (dev.ioctl(DRM_IOCTL_MODE_GETCONNECTOR, &conn_req) < 0) {
                return modes;
            }

            if (conn_req.count_modes == 0) return modes;

            std::vector<struct drm_mode_modeinfo> raw_modes(conn_req.count_modes);
            std::vector<uint32_t> raw_encoders(conn_req.count_encoders);
            std::vector<uint32_t> raw_props(conn_req.count_props);
            std::vector<uint64_t> raw_prop_values(conn_req.count_props);

            conn_req.modes_ptr = reinterpret_cast<uint64_t>(raw_modes.data());
            conn_req.encoders_ptr = reinterpret_cast<uint64_t>(raw_encoders.data());
            conn_req.props_ptr = reinterpret_cast<uint64_t>(raw_props.data());
            conn_req.prop_values_ptr =
↪ reinterpret_cast<uint64_t>(raw_prop_values.data());

            if (dev.ioctl(DRM_IOCTL_MODE_GETCONNECTOR, &conn_req) == 0) {
                for (auto const& raw_mode : raw_modes) {
                    if (raw_mode.hdisplay && raw_mode.vdisplay) {
                        modes.push_back(std::make_unique<mode>(raw_mode));
                    }
                }
            }

            return modes;
        }
    }
}
```

```
    }

    bool connector::apply_config(crtc& c, framebuffer const& fb, mode const& m)
    ↪  const {
        return fb.apply_config(c, connector_id, m);
    }
}

}
```

rendering/renderer-2d.cpp

```
#include "renderer-2d.h"

namespace rendering {

    renderer_2d::renderer_2d(rendering_backend &tgt, assets::asset_manager const& tm,
    ↪ assets::texture const& font_tex) : target(tgt), tex_manager(tm),
    ↪ font_texture(font_tex) {}

    void renderer_2d::draw_texture(assets::texture const& tex, int x, int y, int w,
    ↪ int h) const {
        int const screen_w = static_cast<int>(target("width"_f));
        int const screen_h = static_cast<int>(target("height"_f));

        unsigned int const pitch = target("pitch"_f) / sizeof(std::uint32_t);
        std::uint32_t *__restrict mmio = target("mmio"_f);

        int const start_x = std::max(0, x);
        int const start_y = std::max(0, y);
        int const end_x = std::min(screen_w, x + w);
        int const end_y = std::min(screen_h, y + h);

        std::uint32_t const MAGENTA_COLORKEY = 0xFFFF00FF;

        for (int dx = start_x; dx < end_x; ++dx) {
            std::uint32_t const src_x = ((dx - x) * tex("width"_f)) / w;
            for (int dy = start_y; dy < end_y; ++dy) {
                std::uint32_t const src_y = ((dy - y) * tex("height"_f)) / h;

                std::uint32_t const pixel = tex("pixels"_f)[src_x * tex("height"_f) +
                ↪ src_y];

                if (tex("has_transparency"_f) && pixel == MAGENTA_COLORKEY) {
                    continue;
                }

                mmio[dy * pitch + dx] = pixel;
            }
        }
    }

    void renderer_2d::draw_text(std::string_view text, int x, int y, int char_w, int
    ↪ char_h, std::uint32_t const color) const {
        int const cols = 16;
        int const rows = 16;
        int const src_char_w = font_texture("width"_f) / cols;
        int const src_char_h = font_texture("height"_f) / rows;

        int current_x = x;
        std::uint32_t const MAGENTA_COLORKEY = 0xFFFF00FF;

        for (char const c : text) {
            if (c == '\\n') {
                current_x = x;
                y += char_h;
                continue;
            }
        }
    }

}
```

```

    unsigned char const char_idx = static_cast<unsigned char>(c);
    int const grid_x = char_idx % cols;
    int const grid_y = char_idx / rows;

    int const start_x = std::max(0, current_x);
    int const start_y = std::max(0, y);
    int const end_x = std::min(static_cast<int>(target("width"_f)),
        ↪ current_x + char_w);
    int const end_y = std::min(static_cast<int>(target("height"_f)), y +
        ↪ char_h);

    unsigned int const pitch = target("pitch"_f) / sizeof(std::uint32_t);
    std::uint32_t *__restrict mmio = target("mmio"_f);

    for (int dx = start_x; dx < end_x; ++dx) {
        std::uint32_t const src_x = grid_x * src_char_w + ((dx - current_x) *
            ↪ src_char_w) / char_w;
        for (int dy = start_y; dy < end_y; ++dy) {
            std::uint32_t const src_y = grid_y * src_char_h + ((dy - y) *
                ↪ src_char_h) / char_h;

            std::uint32_t const pixel = font_texture("pixels"_f)[src_x *
                ↪ font_texture("height"_f) + src_y];

            if (font_texture("has_transparency"_f) && pixel ==
                ↪ MAGENTA_COLORKEY) {
                continue;
            }

            mmio[dy * pitch + dx] = pixel * color;
        }
    }

    current_x += char_w;
}

}

void renderer_2d::draw_rect(int x, int y, int w, int h, std::uint32_t color) const
    ↪ {
    auto* fb = target("mmio"_f);
    unsigned pitch = target("pitch"_f) / sizeof(std::uint32_t);
    unsigned scr_w = target("width"_f);
    unsigned scr_h = target("height"_f);
    int x1 = std::max(0, x), y1 = std::max(0, y);
    int x2 = std::min((int)scr_w, x + w), y2 = std::min((int)scr_h, y + h);

    std::uint32_t alpha = (color >> 24) & 0xFF;
    if (alpha == 0xFF) {
        for (int row = y1; row < y2; ++row)
            for (int col = x1; col < x2; ++col)
                fb[row * pitch + col] = color;
    } else {
        std::uint32_t inv = 255 - alpha;
        std::uint32_t sr = (color >> 16) & 0xFF;
        std::uint32_t sg = (color >> 8) & 0xFF;
        std::uint32_t sb = (color >> 0) & 0xFF;
        for (int row = y1; row < y2; ++row) {
            for (int col = x1; col < x2; ++col) {

```

```
std::uint32_t dst = fb[row * pitch + col];  
std::uint32_t r = (sr * alpha + ((dst >> 16) & 0xFF) * inv) / 255;  
std::uint32_t g = (sg * alpha + ((dst >> 8) & 0xFF) * inv) / 255;  
std::uint32_t b = (sb * alpha + ((dst      ) & 0xFF) * inv) / 255;  
fb[row * pitch + col] = (0xFF << 24) | (r << 16) | (g << 8) | b;  
  
}  
  
}  
  
}
```

rendering/visplane.cpp

```
#include "visplane.h"

#include <cmath>
#include <algorithm>
#include <climits>

#include "lighting.h"
#include "frd.h"

rendering::visplane::visplane(unsigned int const sw, float const h, assets::texture_id
↪ const tid, std::uint8_t const light) :
    height(h),
    tex_id(tid),
    light_level(light),
    min_x(INT_MAX),
    max_x(INT_MIN) {
    top.assign(sw, -1);
    bottom.assign(sw, -1);
}

void rendering::visplane::add_column(std::vector<visplane>& pool, int x, int y_start,
↪ int y_end, unsigned int sw, float flat_height, assets::texture_id tex_id,
↪ std::uint8_t light_level) {
    if (y_start >= y_end) return;

    visplane* target_vp = nullptr;

    /* try to find matching visplane */
    for (auto& vp : pool) {
        if (vp.height != flat_height || vp.tex_id != tex_id || vp.light_level !=
↪ light_level)
            continue;
        /* can merge if column empty or touches existing column */
        if (vp.top[x] == -1 || (y_start <= vp.bottom[x] && y_end >= vp.top[x])) {
            target_vp = &vp;
            break;
        }
    }

    /* if none found, create a new visplane */
    if (!target_vp) {
        pool.emplace_back(sw, flat_height, tex_id, light_level);
        target_vp = &pool.back();
    }

    /* update the visplane bounds */
    target_vp->min_x = std::min(target_vp->min_x, x);
    target_vp->max_x = std::max(target_vp->max_x, x);

    if (target_vp->top[x] == -1) {
        target_vp->top[x] = y_start;
        target_vp->bottom[x] = y_end;
    } else {
        target_vp->top[x] = std::min(target_vp->top[x], y_start);
        target_vp->bottom[x] = std::max(target_vp->bottom[x], y_end);
    }
}
```

```
}
```

```
void rendering::visplane::render(assets::asset_manager const& tex_manager,
↳ frame_rendering_data const& frd) const {
    /* optimizations */
    float dx_cos = frd.inv_fov_scale * frd.cos_cam_angle;
    float dx_sin = frd.inv_fov_scale * frd.sin_cam_angle;

    assets::texture const& tex = tex_manager.flat_tx_by_id(tex_id);
    float h = std::abs(frd.cam_height - height);

    /* true y bounds */
    int min_y = frd.sh, max_y = 0;
    for (int x = min_x; x <= max_x; ++x) {
        if (top[x] != -1) {
            min_y = std::min(min_y, (int)top[x]);
            max_y = std::max(max_y, (int)bottom[x]);
        }
    }

    /* draw loop; line by line */
    for (int y = min_y; y < max_y; ++y) {
        int dy = std::abs(y - (int)(frd.sh / 2));
        if (dy == 0) continue;

        /* constant depth z */
        float z = (h * frd.fov_scale) / (float)dy;
        /* because of that, constant light */
        /* also, no fisheye effect correction on flat lighting bc it would */
        /* be too expensive to calc lighting per-pixel */
        int row_light = lighting::calculate(light_level, z);

        /* steps in world space as fpxt 16.16*/
        int32_t step_x = (int32_t)(z * dx_cos * 65536.0f);
        int32_t step_y = (int32_t)(z * dx_sin * 65536.0f);

        /* scan row to find spans */
        int x = min_x;
        while (x <= max_x) {
            /* record span */
            while (x <= max_x && (top[x] == -1 || y < top[x] || y >= bottom[x])) {
                x++;
            }
            int span_start = x;

            /* find end of span */
            while (x <= max_x && (top[x] != -1 && y >= top[x] && y < bottom[x])) {
                x++;
            }
            int span_end = x;

            /* span ended, draw */

            float px = (float)span_start - frd.half_sw;
            /* grab world space coords */
            float world_x_f = z * (px * frd.inv_fov_scale * frd.cos_cam_angle -
↳ frd.sin_cam_angle) + frd.cam_pos("x"_f);
            float world_y_f = z * (px * frd.inv_fov_scale * frd.sin_cam_angle +
↳ frd.cos_cam_angle) + frd.cam_pos("y"_f);
```



```

    /* once again world coords as fxt 16.16 to avoid fp math */
    int32_t fx = (int32_t)(world_x_f * 65536.0f);
    int32_t fy = (int32_t)(world_y_f * 65536.0f);

    /* current line ptr */
    uint32_t* dest = &frd.mmio[span_start + y * frd.pitch];

    for (int sx = span_start; sx < span_end; ++sx) {
        /* texture coords */
        int tx = ((fx >> 16) % tex("width"_f));
        int ty = ((fy >> 16) % tex("height"_f));
        /* ensure they are positive (modulo can be negative) */
        if (tx < 0) tx += tex("width"_f);
        if (ty < 0) ty += tex("height"_f);

        /* write the pixel */
        std::uint32_t color = tex("pixels"_f)[tx * tex("height"_f) + ty];
        *dest++ = lighting::apply(color, row_light);

        /* update world coords */
        fx += step_x;
        fy += step_y;
    }
}
}
}

```

rendering/software-renderer.cpp

```
#include "software-renderer.h"

#include <cmath>
#include <algorithm>
#include <vector>
#include <utility>

#include "lighting.h"
#include "geometry/sidedef.h"
#include "geometry/sector.h"
#include "geometry/subsector.h"

namespace rendering {

software_renderer::software_renderer(rendering_backend &tgt, assets::asset_manager
→   const& tm, geometry::map_data const& map)
    : target(tgt), tex_manager(tm), current_map(map) {}

void software_renderer::render_bsp_node(util::indexed_storage<geometry::bsp_node>::
→   id_t node_id, frame_rendering_data const& frd) {
    /* if it's a leaf, then it's a subsector, so we should render the linedefs */
    if (geometry::bsp_node::is_leaf(node_id)) {
        auto subsector_id = geometry::bsp_node::get_id(node_id);

        geometry::subsector const& sub = current_map.subsectors[subsector_id];

        std::uint8_t sub_light = 255;
        if (!sub.lines.empty()) {
            auto const& first_line = current_map.linedefs[sub.lines[0]];
            sub_light = current_map.sectors[current_map.sidedefs[first_line.front].
→   facing_sector].light_level;
        }
        for(auto const& sprite : sub.sprites) {
            add_vissprite(sprite, sub_light, frd);
        }

        for (auto const& line_id : sub.lines) {
            project_and_draw_linedef(current_map.linedefs[line_id], frd);
        }
        return;
    }

    /* if not a leaf, then a bsp node */
    geometry::bsp_node const& node = current_map.nodes[node_id];

    /* check if camera is in front */
    bool is_front = node.is_pt_front_side(frd.cam_pos);

    /* identify closer and farther nodes and their bounding boxes */
    auto closer_node = is_front ? node.front : node.back;
    auto farther_node = is_front ? node.back : node.front;
    // auto closer_box = is_front ? node.front_box : node.back_box;
    auto farther_box = is_front ? node.back_box : node.front_box;

    /* visit the closer node first */

```

```

// if (is_box_visible(closer_box, frd)) {
//     render_bsp_node(closer_node, frd);
// }

/* visit the farther node second */
if (is_box_visible(farther_box, frd)) {
    render_bsp_node(farther_node, frd);
}
}

void software_renderer::project_and_draw_linedef(geometry::linedef line,
↪ frame_rendering_data const& frd) {
    math::vec2 tr_v1 = line("seg"_f)("point0"_f) - frd.cam_pos;
    math::vec2 tr_v2 = line("seg"_f)("point1"_f) - frd.cam_pos;

    tr_v1 = math::vec2::rotate_with_known_trig(tr_v1, frd.cos_cam_angle,
↪ -frd.sin_cam_angle);
    tr_v2 = math::vec2::rotate_with_known_trig(tr_v2, frd.cos_cam_angle,
↪ -frd.sin_cam_angle);

    /* now translated v1 & v2 are exactly opposite the camera */

    /* u coordinates for textures */
    float u1 = 0.0f;
    float u2 = line.len();

    /* return if behind camera */
    if (tr_v1("y"_f) <= near_z && tr_v2("y"_f) <= near_z) return;

    /* clip against the near plane */
    float inv_dy = 1.0f / (tr_v2("y"_f) - tr_v1("y"_f));
    if (tr_v1("y"_f) < near_z) {
        float t = (near_z - tr_v1("y"_f)) * inv_dy;
        tr_v1("x"_f) = tr_v1("x"_f) + t * (tr_v2("x"_f) - tr_v1("x"_f));
        tr_v1("y"_f) = near_z;
        u1 = u1 + t * (u2 - u1);
    } else if (tr_v2("y"_f) < near_z) {
        float t = (tr_v2("y"_f) - near_z) * inv_dy;
        tr_v2("x"_f) = tr_v2("x"_f) + t * (tr_v1("x"_f) - tr_v2("x"_f));
        tr_v2("y"_f) = near_z;
        u2 = u2 + t * (u1 - u2);
    }

    /* exact values for tex mapping */
    float proj_x1 = (tr_v1("x"_f) / tr_v1("y"_f)) * frd.fov_scale + frd.half_sw;
    float proj_x2 = (tr_v2("x"_f) / tr_v2("y"_f)) * frd.fov_scale + frd.half_sw;

    /* handle back-face viewing */
    if (proj_x1 > proj_x2) {
        /* cull back face (wall facing opposite side) */
        if (line.is_wall()) {
            return;
        }

        /* swap everything */
        std::swap(proj_x1, proj_x2);
        std::swap(tr_v1, tr_v2);
        std::swap(u1, u2);
        std::swap(line.front, line.back);
    }
}

```

```

    }

    if (line.is_wall()) {
        draw_solid_wall_span(proj_x1, proj_x2, tr_v1("y"_f), tr_v2("y"_f), u1, u2,
↪ line, frd);
    } else {
        draw_portal_wall_span(proj_x1, proj_x2, tr_v1("y"_f), tr_v2("y"_f), u1, u2,
↪ line, frd);
    }
}

void software_renderer::draw_solid_wall_span(float proj_x1, float proj_x2, float z1,
↪ float z2, float u1, float u2, geometry::linedef const& line, frame_rendering_data
↪ const& frd) {
    /* useful stuff */
    geometry::sidedef const& sd = current_map.sidedefs[line.front];
    geometry::sector const& s = current_map.sectors[sd.facing_sector];
    assets::texture const& mt = tex_manager.wall_tx_by_id(sd.middle_tex);

    /* bounds as integers */
    int x1 = static_cast<int>(proj_x1);
    int x2 = static_cast<int>(proj_x2);
    float inv_z1 = 1.0f / z1;
    float inv_z2 = 1.0f / z2;
    /* clamped x1 and x2 */
    int cx1 = std::max(0, x1);
    int cx2 = std::min((int)frd.sw, x2);

    for (int x = cx1; x < cx2; ++x) {
        /* done already */
        if (lower_clip[x] <= upper_clip[x]) continue;

        /* t ∈ [0, 1) */
        float t = (x - proj_x1) / (proj_x2 - proj_x1);
        /* perspective correction with depth */
        float inv_z = (1.0f - t) * inv_z1 + t * inv_z2;
        float depth = 1.0f / inv_z;

        /* texture u coord */
        float u_over_z = (1.0f - t) * u1 * inv_z1 + t * u2 * inv_z2;
        unsigned int u = (unsigned int)(u_over_z * depth) % mt("width"_f);

        /* light depends on depth */
        int column_light = lighting::calculate(s.light_level, depth *
↪ (*frd.euclidian_dist_factor)[x]);

        /* project y onto the screen */
        int top_y = frd.sh/2 - (int)((s.ceiling_height - frd.cam_height) *
↪ frd.fov_scale * inv_z);
        int bot_y = frd.sh/2 - (int)((s.floor_height - frd.cam_height) * frd.fov_scale
↪ * inv_z);

        /* crop to undendered space */
        int cropped_top_y = std::max(top_y, std::max(0, upper_clip[x]));
        int cropped_bot_y = std::min(bot_y, lower_clip[x]);

        /* draw ceiling from upper clip to wall top */
        visplane::add_column(visplanes, x, std::max(0, upper_clip[x]), cropped_top_y,
↪ frd.sw, s.ceiling_height, s.ceiling_tex, s.light_level);
    }
}

```

```

        /* draw floor from wall bottom to lower clip */
        visplane::add_column(visplanes, x, cropped_bot_y, lower_clip[x], frd.sw,
→ s.floor_height, s.floor_tex, s.light_level);

        /* v scales linearly so pre-calculate step to avoid div */
        float v_step = (float)mt("height"_f) / (float)(bot_y - top_y);
        float current_v = (float)(cropped_top_y - top_y) * v_step;

        /* draw loop */
        for (int y = cropped_top_y; y < cropped_bot_y; ++y) {
            /* texture v coord */
            unsigned int v = static_cast<unsigned int>(current_v);
            current_v += v_step;

            /* write the pixel */
            std::uint32_t color = mt("pixels"_f)[u * mt("height"_f) + v];
            frd.mmio[x + y * frd.pitch] = lighting::apply(color, column_light);
        }

        /* update clipping */
        upper_clip[x] = frd.sh;
        lower_clip[x] = 0;
    }
}

void software_renderer::draw_portal_wall_span(float proj_x1, float proj_x2, float z1,
→ float z2, float u1, float u2, geometry::linedef const& line, frame_rendering_data
→ const& frd) {
    /* useful stuff */
    geometry::sidedef const& front_sd = current_map.sidedefs[line.front];
    geometry::sidedef const& back_sd = current_map.sidedefs[line.back];
    geometry::sector const& front = current_map.sectors[front_sd.facing_sector];
    geometry::sector const& back = current_map.sectors[back_sd.facing_sector];
    assets::texture const& ut = tex_manager.wall_tx_by_id(front_sd.upper_tex);
    assets::texture const& lt = tex_manager.wall_tx_by_id(front_sd.lower_tex);
    float inv_z1 = 1.0f / z1;
    float inv_z2 = 1.0f / z2;

    /* bounds as integers */
    int x1 = (int)proj_x1;
    int x2 = (int)proj_x2;
    /* clamped x1 and x2 */
    int cx1 = std::max(0, x1);
    int cx2 = std::min((int)frd.sw, x2);

    for (int x = cx1; x < cx2; ++x) {
        /* done already */
        if (lower_clip[x] <= upper_clip[x]) continue;

        /* t E [0, 1) */
        float t = ((float)x - proj_x1) / (proj_x2 - proj_x1);
        /* perspective correction with depth */
        float inv_z = (1.0f - t) * inv_z1 + t * inv_z2;
        float depth = 1.0f / inv_z;

        /* texture u coord */
        float u_over_z = (1.0f - t) * u1 * inv_z1 + t * u2 * inv_z2;
        unsigned int u = (unsigned int)(u_over_z * depth);
        unsigned int u_ut = u % ut("width"_f);

```

```

unsigned int u_lt = u % lt("width"_f);

/* light depends on depth */
int column_light = lighting::calculate(front.light_level, depth *
↳ (*frd.euclidian_dist_factor)[x]);

/* project front ceil, front floor, back ceil, back floor onto the screen */
int c_fc = frd.sh/2 - (int)((front.ceiling_height - frd.cam_height) *
↳ frd.fov_scale * inv_z);
int c_ff = frd.sh/2 - (int)((front.floor_height - frd.cam_height) *
↳ frd.fov_scale * inv_z);
int c_bc = frd.sh/2 - (int)((back.ceiling_height - frd.cam_height) *
↳ frd.fov_scale * inv_z);
int c_bf = frd.sh/2 - (int)((back.floor_height - frd.cam_height) *
↳ frd.fov_scale * inv_z);

/* draw ceiling from upper clip to wall top */
visplane::add_column(visplanes, x, std::max(0, upper_clip[x]), std::max(c_fc,
↳ upper_clip[x]), frd.sw, front.ceiling_height, front.ceiling_tex,
↳ front.light_level);
/* draw floor from wall bottom to lower clip */
visplane::add_column(visplanes, x, std::min(c_ff, lower_clip[x]),
↳ lower_clip[x], frd.sw, front.floor_height, front.floor_tex, front.light_level);

/* draw loop if the front ceil has lower y than the back ceil (is higher) */
if (c_fc < c_bc) {
    int draw_top = std::max(c_fc, std::max(0, upper_clip[x]));
    int draw_bot = std::min(c_bc, lower_clip[x]);

    /* v scales linearly so pre-calculate step to avoid div */
    float v_step = (float)ut("height"_f) / (float)(c_bc - c_fc);
    float current_v = (float)(draw_top - c_fc) * v_step;

    for (int y = draw_top; y < draw_bot; ++y) {
        /* texture v coord */
        unsigned int v = static_cast<unsigned int>(current_v);
        current_v += v_step;

        /* write the pixel */
        std::uint32_t color = ut("pixels"_f)[u_lt * ut("height"_f) + v];
        frd.mmio[x + y * frd.pitch] = lighting::apply(color, column_light);
    }
}

/* same thing, but if the front floor is world lower than back floor (has
↳ higher y) */
if (c_ff > c_bf) {
    int draw_top = std::max(c_bf, std::max(0, upper_clip[x]));
    int draw_bot = std::min(c_ff, lower_clip[x]);

    /* v scales linearly so pre-calculate step to avoid div */
    float v_step = (float)lt("height"_f) / (float)(c_ff - c_bf);
    float current_v = (float)(draw_top - c_bf) * v_step;

    for (int y = draw_top; y < draw_bot; ++y) {
        unsigned int v = static_cast<unsigned int>(current_v);
        current_v += v_step;

        std::uint32_t color = lt("pixels"_f)[u_lt * lt("height"_f) + v];

```

```

        frd.mmio[x + y * frd.pitch] = lighting::apply(color, column_light);
    }
}

    /* update clipping */
    upper_clip[x] = std::max(upper_clip[x], std::max(c_fc, c_bc));
    lower_clip[x] = std::min(lower_clip[x], std::min(c_ff, c_bf));
}
}

void software_renderer::render_visplanes(frame_rendering_data const& frd) {
    for (auto const& vp : visplanes)
        vp.render(tex_manager, frd);
    visplanes.clear();
}

void software_renderer::add_vissprite(sprite *const s, std::uint8_t light,
↪ frame_rendering_data const& frd) {
    math::vec2 tr_pos = (*s)("pos"_f) - frd.cam_pos;
    tr_pos = math::vec2::rotate_with_known_trig(tr_pos, frd.cos_cam_angle,
↪ -frd.sin_cam_angle);

    if(tr_pos("y"_f) <= near_z) return;

    float proj_x = (tr_pos("x"_f) / tr_pos("y"_f)) * frd.fov_scale + frd.half_sw;
    float scale = frd.fov_scale / tr_pos("y"_f) * s->inherent_scale;

    assets::texture const& tex = tex_manager.sprite_tx_by_id(s->tex_id);

    /* screen bounds */
    float half_width = (tex("width"_f) * scale) / 2.0f;
    int x1 = static_cast<int>(proj_x - half_width);
    int x2 = static_cast<int>(proj_x + half_width);

    /* frustum culling */
    if(x1 >= static_cast<int>(frd.sw) || x2 < 0) return;

    int cx1 = std::max(0, x1);
    int cx2 = std::min(x2, static_cast<int>(frd.sw) - 1);

    /* occlusion culling */
    bool visible = false;
    for(int x = cx1; x <= cx2; ++x) {
        if(lower_clip[x] > upper_clip[x]) {
            visible = true;
            break;
        }
    }
    if(!visible) return;

    std::vector<int> slice_uc(upper_clip.begin() + cx1, upper_clip.begin() + cx2 + 1);
    std::vector<int> slice_lc(lower_clip.begin() + cx1, lower_clip.begin() + cx2 + 1);
    vissprites.emplace_back(*s, tr_pos("y"_f), cx1, cx2, proj_x, scale, light,
↪ std::move(slice_uc), std::move(slice_lc));
}

void software_renderer::render_vissprites(frame_rendering_data const& frd) {
    vissprite::pa_sort(vissprites);
}

```

```

    for (auto const& vs : vissprites)
        vs.render(tex_manager, frd);

    vissprites.clear();
}

bool software_renderer::is_box_visible(geometry::bsp_node::bounding_box const& box,
↳ frame_rendering_data const& frd) {
    /* if the camera is inside, we def need to render the box */
    if (frd.cam_pos("x"_f) >= box.left && frd.cam_pos("x"_f) <= box.right &&
        frd.cam_pos("y"_f) >= box.bottom && frd.cam_pos("y"_f) <= box.top) {
        return true;
    }

    /* corners of the bounding box */
    math::vec2 corners[4] = {
        {box.left, box.top},
        {box.right, box.top},
        {box.right, box.bottom},
        {box.left, box.bottom}
    };

    /* transform corners to camera space */
    for (int i = 0; i < 4; ++i) {
        corners[i] = corners[i] - frd.cam_pos;
        corners[i] = math::vec2::rotate_with_known_trig(corners[i], frd.cos_cam_angle,
↳ -frd.sin_cam_angle);
    }

    /* clipping a rectangle against a line produces at most 5 vertices */
    math::vec2 clipped[5];
    int num_clipped = 0;

    for (int i = 0; i < 4; ++i) {
        math::vec2 p1 = corners[i];
        /* modulo 4, but faster */
        math::vec2 p2 = corners[(i + 1) & 3];

        bool p1_inside = p1("y"_f) >= near_z;
        bool p2_inside = p2("y"_f) >= near_z;

        if (p1_inside) {
            clipped[num_clipped++] = p1;
        }

        if (p1_inside != p2_inside) {
            /* edge intersects the near plane, calculate intersection */
            float t = (near_z - p1("y"_f)) / (p2("y"_f) - p1("y"_f));
            clipped[num_clipped++] = { p1("x"_f) + t * (p2("x"_f) - p1("x"_f)), near_z
↳ };
        }
    }

    /* entirely behind the near plane, cull */
    if (num_clipped == 0) return false;

    /* project the clipper verts onto the screen */
    float min_proj_x = 1e9f;
    float max_proj_x = -1e9f;

```



```

for (int i = 0; i < num_clipped; ++i) {
    float proj_x = (clipped[i]("x"_f) / clipped[i]("y"_f)) * frd.fov_scale +
        ↪ frd.half_sw;
    min_proj_x = std::min(min_proj_x, proj_x);
    max_proj_x = std::max(max_proj_x, proj_x);
}

int min_x = static_cast<int>(min_proj_x);
int max_x = static_cast<int>(max_proj_x);

/* frustum culling (is outside of FOV?) */
if (min_x >= static_cast<int>(frd.sw) || max_x < 0) {
    return false;
}

/* clamp to the parts visible on screen */
int cx1 = std::max(0, min_x);
int cx2 = std::min(static_cast<int>(frd.sw) - 1, max_x);

/* occlusion culling (fully behind walls) */
for (int x = cx1; x <= cx2; ++x) {
    if (lower_clip[x] > upper_clip[x]) {
        ↪ /* at least one column is open, so it's partially visible */
        return true;
    }
}

/* fully occluded by walls */
return false;
}

void software_renderer::render_bsp(math::vec2 const cam_pos, float const cam_height,
    ↪ float cam_angle, float fov) {

    if (current_map.root_node_id != util::indexed_storage<geometry::bsp_node>::nullid)
        ↪ {
        ↪ /* pre-calculate a bunch of things because calculating them per-pixel or even
        ↪ ↪ per-row is too slow */
        frame_rendering_data frd;
        frd.cam_pos = cam_pos;
        frd.cam_height = cam_height;
        frd.cam_angle = cam_angle;
        frd.sw = target("width"_f);
        frd.sh = target("height"_f);
        frd.half_sw = frd.sw / 2.0f;
        frd.pitch = target("pitch"_f)/sizeof(std::uint32_t);
        frd.mmio = target("mmio"_f);
        frd.fov_scale = frd.half_sw / std::tan(fov/2.0f);
        frd.inv_fov_scale = 1.0f / frd.fov_scale;
        frd.cos_cam_angle = std::cos(frd.cam_angle);
        frd.sin_cam_angle = std::sin(frd.cam_angle);

        upper_clip.assign(frd.sw, -1);
        lower_clip.assign(frd.sw, frd.sh);

        if (euclidian_dist_factor.size() != frd.sw) {
            euclidian_dist_factor.resize(frd.sw);
            float center_x = frd.half_sw;
            for (unsigned int x = 0; x < frd.sw; ++x) {

```

```
        float dx = (x - center_x) * frd.inv_fov_scale;
        euclidian_dist_factor[x] = std::sqrt(dx * dx + 1.0f);
    }
}
frd.euclidian_dist_factor = &euclidian_dist_factor;

render_bsp_node(current_map.root_node_id, frd);

render_visplanes(frd);
render_vissprites(frd);
}
}
}
```

rendering/sprite.cpp

```
#include "sprite.h"
```

```
rendering::sprite::sprite(math::vec2 const p, float const z, assets::texture_id const  
↪ tex, float const is) :  
    pos(p),  
    z_pos(z),  
    angle(0.0f),  
    tex_id(tex),  
    inherent_scale(is) { }
```

rendering/vissprite.cpp

```
#include "vissprite.h"

#include <algorithm>

#include "lighting.h"
#include "frd.h"

rendering::vissprite::vissprite(sprite const& sprite, float const z, int const
↳ clamped_x1, int const clamped_x2, float const px, float const sc, std::uint8_t
↳ const light, std::vector<int> const& uc, std::vector<int> const& lc) :
    depth(z),
    cx1(clamped_x1),
    cx2(clamped_x2),
    proj_x(px),
    scale(sc),
    z_pos(sprite.z_pos),
    tex_id(sprite.tex_id),
    light_level(light),
    flash_red(sprite.hit_flash > 0.0f),
    upper_clip(uc),
    lower_clip(lc) { }

void rendering::vissprite::pa_sort(std::vector<vissprite> &vec) {
    std::sort(vec.begin(), vec.end(), [](vissprite const& a, vissprite const& b) {
        return a.depth > b.depth;
    });
}

void rendering::vissprite::render(assets::asset_manager const& tex_manager,
↳ frame_rendering_data const& frd) const {
    assets::texture const& tex = tex_manager.sprite_tx_by_id(tex_id);
    float eu_dist_f =
        ↳ (*frd.euclidian_dist_factor)[std::clamp(static_cast<int>(proj_x), 0,
        ↳ static_cast<int>(frd.sw) - 1)];
    int sprite_light = lighting::calculate(light_level, depth * eu_dist_f);

    float scr_y_bot = frd.sh/2.0f - (z_pos - frd.cam_height) * scale;
    float scr_y_top = scr_y_bot - tex("height"_f) * scale;

    float inv_scale = 1.0f / scale;

    for(int x = cx1; x <= cx2; ++x) {
        int clip_top = upper_clip[x - cx1];
        int clip_bot = lower_clip[x - cx1];

        /* occlusion culling */
        if(clip_bot <= clip_top) continue;

        float draw_top = std::max(static_cast<float>(std::max(0, clip_top)),
        ↳ scr_y_top);
        float draw_bot = std::min(static_cast<float>(clip_bot), scr_y_bot);

        if(draw_bot <= draw_top) continue;

        /* should be correct ??? */
        int u = static_cast<int>((x - proj_x + (tex("width"_f)*scale)/2) * inv_scale);
```

```

u = std::clamp(u, 0, static_cast<int>(tex("width"_f)) - 1);

float current_v = (draw_top - scr_y_top) * inv_scale;

for(int y = static_cast<int>(draw_top); y < static_cast<int>(draw_bot); ++y) {
    int v = static_cast<int>(current_v);
    current_v += inv_scale;

    std::uint32_t color = tex("pixels"_f)[u * tex("height"_f) + v];
    if(color != 0xffff00ff) {
        std::uint32_t lit = lighting::apply(color, sprite_light);
        if (flash_red) {
            std::uint32_t r = std::min(255u, ((lit >> 16) & 0xFF) + 180u);
            std::uint32_t g = ((lit >> 8) & 0xFF) / 3;
            std::uint32_t b = (lit & 0xFF) / 3;
            lit = (lit & 0xFF000000) | (r << 16) | (g << 8) | b;
        }
        frd.mmio[x + y * frd.pitch] = lit;
    }
}
}
}

```

util/binary-resource.cpp

```
#include "binary-resource.h"

namespace util {
    void const* binary_resource::operator()(component_tag<"beginning">) const {
        return begin_ptr;
    }

    void const* binary_resource::operator()(component_tag<"end">) const {
        return end_ptr;
    }

    std::size_t binary_resource::operator()(component_tag<"size">) const {
        return byte_size;
    }

    binary_resource::binary_resource(void const* begin, void const* end) :
        begin_ptr(begin),
        end_ptr(end),
        byte_size(static_cast<std::size_t>(static_cast<std::byte const*>(end) -
↪ static_cast<std::byte const*>(begin)))
    {}
}
```

util/file-resource.cpp

```
#include "file-resource.h"

namespace util {
    void const* file_resource::operator()(component_tag<"beginning">) const {
        return file_data.data();
    }

    void const* file_resource::operator()(component_tag<"end">) const {
        return file_data.data() + file_data.size();
    }

    std::size_t file_resource::operator()(component_tag<"size">) const {
        return file_data.size();
    }

    file_resource::file_resource(std::vector<std::byte> data) :
        resource(),
        file_data(std::move(data))
    {}
}
```

util/resource-loader.cpp

```
#include "resource-loader.h"

#include <dlfcn.h>
#include <filesystem>
#include <fstream>
#include <vector>

namespace util {
    void* resource_loader::lookup_binary(std::string_view resource_name) {
        std::string symbol_name = make_symbol_name(resource_name);

        {
            std::shared_lock<std::shared_mutex> rlock(cache_mutex);
            if (auto it = symbol_cache.find(symbol_name); it != symbol_cache.end()) {
                return it->second;
            }
        }

        void* ptr = dlsym(RTLD_DEFAULT, symbol_name.c_str());

        {
            std::unique_lock<std::shared_mutex> wlock(cache_mutex);
            symbol_cache.emplace(symbol_name, ptr);
        }

        return ptr;
    }

    std::unique_ptr<resource> resource_loader::load_file(std::string_view
    ↪ resource_name) const {
        std::filesystem::path file_path = std::filesystem::path("./res") /
        ↪ resource_name;

        if (std::filesystem::exists(file_path) &&
        ↪ std::filesystem::is_regular_file(file_path)) {
            std::ifstream file(file_path, std::ios::binary | std::ios::ate);
            if (file) {
                std::streamsize size = file.tellg();
                file.seekg(0, std::ios::beg);

                std::vector<std::byte> buffer(static_cast<std::size_t>(size));
                if (file.read(reinterpret_cast<char*>(buffer.data()), size)) {
                    return std::make_unique<file_resource>(std::move(buffer));
                }
            }
        }

        return nullptr;
    }

    resource* resource_loader::lookup_resource_runtime(std::string_view resource_name)
    ↪ {
        std::string res_name_str(resource_name);

        {
            std::shared_lock<std::shared_mutex> rlock(cache_mutex);
```



```

        if (auto it = resource_cache.find(res_name_str); it !=
            resource_cache.end()) {
            return it->second.get();
        }
    }

    if (std::unique_ptr<resource> file_res = load_file(resource_name)) {
        std::unique_lock<std::shared_mutex> wlock(cache_mutex);
        resource* ptr = file_res.get();
        resource_cache.emplace(res_name_str, std::move(file_res));
        return ptr;
    }

    std::string begin_name = res_name_str + "_start";
    std::string end_name   = res_name_str + "_end";

    void* begin_ptr = lookup_binary(begin_name);
    void* end_ptr   = lookup_binary(end_name);

    if (begin_ptr && end_ptr) {
        auto sym_res = std::make_unique<binary_resource>(begin_ptr, end_ptr);
        std::unique_lock<std::shared_mutex> wlock(cache_mutex);
        resource* ptr = sym_res.get();
        resource_cache.emplace(res_name_str, std::move(sym_res));
        return ptr;
    }

    return nullptr;
}

}

```

assets/asset-manager.cpp

```
#include "asset-manager.h"
#include "rendering/renderer-2d.h"
#include "rendering/rendering-backend.h"

#include <sstream>

namespace assets {

    asset_manager::asset_manager(util::resource_loader &resld,
    ↪ std::vector<asset_pack> packs)
        : rl(resld), asset_packs(std::move(packs)), cur_set(0) {}

    asset_manager asset_manager::load(util::resource_loader &resld) {
        std::vector<asset_pack> packs;

        auto r = resld.lookup_resource("meta-texture-sets");
        if (!r) {
            return asset_manager(resld, packs);
        }

        util::resource &meta_res = *r;

        if (!meta_res("beginning"_f) || meta_res("size"_f) == 0) {
            return asset_manager(resld, packs);
        }

        std::string content(reinterpret_cast<const char*>(meta_res("beginning"_f)),
    ↪ meta_res("size"_f));
        std::istringstream stream(content);
        std::string line;

        while (std::getline(stream, line)) {
            if (!line.empty() && line.back() == '\\r') {
                line.pop_back();
            }
            if (line.empty()) {
                continue;
            }

            packs.push_back(asset_pack::load(resld, line));
        }

        return asset_manager(resld, std::move(packs));
    }

    void asset_manager::cycle_set() {
        if (asset_packs.empty()) {
            return;
        }
        cur_set = (cur_set + 1) % static_cast<unsigned int>(asset_packs.size());
    }

    texture const& asset_manager::wall_tx_by_id(texture_id const id) const {
        return asset_packs[cur_set].wall_tx_by_id(id);
    }
}
```

```

texture const& asset_manager::sprite_tx_by_id(texture_id const id) const {
    return asset_packs[cur_set].sprite_tx_by_id(id);
}

texture const& asset_manager::flat_tx_by_id(texture_id const id) const {
    return asset_packs[cur_set].flat_tx_by_id(id);
}

texture const& asset_manager::ui_tx_by_id(texture_id const id) const {
    return asset_packs[cur_set].ui_tx_by_id(id);
}

menu &asset_manager::menu_by_id(menu_id const id) {
    return asset_packs[cur_set].menu_by_id(id);
}

audio_clip const& asset_manager::audio_clip_by_id(audio_clip_id const id) const {
    return asset_packs[cur_set].audio_clip_by_id(id);
}

int asset_manager::display_menu(menu_id const id, rendering::renderer_2d const&
↪ r2d, rendering::rendering_backend &rb, input::input_backend &in) {
    return asset_packs[cur_set].display_menu(id, r2d, rb, in);
}
}

```

assets/asset-pack.cpp

```
#include "asset-pack.h"
#include <sstream>
#include <string>
#include <unordered_map>

namespace assets {

    asset_pack::asset_pack(std::vector<texture> walls, std::vector<texture> sprites,
↪ std::vector<texture> flats, std::vector<texture> ui, std::vector<menu> ms,
↪ std::vector<audio_clip> clips) :
        wall_textures(std::move(walls)),
        sprite_textures(std::move(sprites)),
        flat_textures(std::move(flats)),
        ui_textures(std::move(ui)),
        menus(std::move(ms)),
        audio_clips(std::move(clips)) {}

    texture const& asset_pack::wall_tx_by_id(texture_id const id) const {
        return wall_textures.at(id);
    }

    texture const& asset_pack::sprite_tx_by_id(texture_id const id) const {
        return sprite_textures.at(id);
    }

    texture const& asset_pack::flat_tx_by_id(texture_id const id) const {
        return flat_textures.at(id);
    }

    texture const& asset_pack::ui_tx_by_id(texture_id const id) const {
        return ui_textures.at(id);
    }

    menu &asset_pack::menu_by_id(menu_id const id) {
        return menus.at(id);
    }

    audio_clip const& asset_pack::audio_clip_by_id(audio_clip_id const id) const {
        return audio_clips.at(id);
    }

    std::vector<texture> asset_pack::tx_from_meta(util::resource_loader &resld,
↪ std::string_view meta_path) {
        auto r = resld.lookup_resource(meta_path);
        if (!r) {
            return {};
        }

        util::resource &meta_res = *r;

        if (!meta_res("beginning"_f) || meta_res("size"_f) == 0) {
            return {};
        }

        std::string content(reinterpret_cast<const char*>(meta_res("beginning"_f)),
↪ meta_res("size"_f));
```

```

std::istringstream stream(content);
std::string line;
std::vector<texture> textures;

while (std::getline(stream, line)) {
    if (!line.empty() && line.back() == '\\r') {
        line.pop_back();
    }
    if (line.empty()) {
        continue;
    }

    auto tr = resld.lookup_resource(line);
    if (tr) {
        textures.push_back(texture::load_from_bin(*tr));
    }
}

return textures;
}

std::vector<menu> asset_pack::menus_from_meta(util::resource_loader &resld,
↳ std::string_view meta_path) {
    auto r = resld.lookup_resource(meta_path);
    if (!r) {
        return {};
    }

    util::resource &meta_res = *r;

    if (!meta_res("beginning"_f) || meta_res("size"_f) == 0) {
        return {};
    }

    std::string content(reinterpret_cast<const char*>(meta_res("beginning"_f)),
↳ meta_res("size"_f));
    std::istringstream stream(content);
    std::string line;
    std::vector<menu> result;

    while (std::getline(stream, line)) {
        if (!line.empty() && line.back() == '\\r') {
            line.pop_back();
        }
        if (line.empty()) {
            continue;
        }

        auto mr = resld.lookup_resource(line);
        if (mr) {
            result.push_back(menu::load_from_bin(resld, *mr));
        }
    }

    return result;
}

std::vector<audio_clip> asset_pack::clips_from_meta(util::resource_loader &resld,
↳ std::string_view meta_path) {

```

```

    auto r = resld.lookup_resource(meta_path);
    if (!r) return {};

    util::resource &meta_res = *r;
    if (!meta_res("beginning"_f) || meta_res("size"_f) == 0) return {};

    std::string content(reinterpret_cast<const char*>(meta_res("beginning"_f)),
        ↪ meta_res("size"_f));
    std::istringstream stream(content);
    std::string line;
    std::vector<audio_clip> clips;

    while (std::getline(stream, line)) {
        if (!line.empty() && line.back() == '\r') line.pop_back();
        if (line.empty()) continue;

        auto cr = resld.lookup_resource(line);
        if (cr) clips.push_back(audio_clip::load_from_bin(*cr));
    }

    return clips;
}

std::unordered_map<std::string, std::string>
↪ asset_pack::parse_meta_keys(std::string const& content) {
    std::unordered_map<std::string, std::string> keys;
    std::istringstream stream(content);
    std::string line;

    while (std::getline(stream, line)) {
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }
        if (line.empty()) {
            continue;
        }

        auto colon = line.find(':');
        if (colon == std::string::npos) {
            continue;
        }

        std::string key = line.substr(0, colon);
        std::string value = line.substr(colon + 1);

        auto trim = [](std::string &s) {
            std::size_t start = s.find_first_not_of(" \t");
            std::size_t end = s.find_last_not_of(" \t");
            s = (start == std::string::npos) ? "" : s.substr(start, end - start +
↪ 1);

        };
        trim(key);
        trim(value);

        if (!key.empty()) {
            keys[key] = value;
        }
    }
}

```

```

        return keys;
    }

    asset_pack asset_pack::load(util::resource_loader &resld, std::string_view
    ↪ pack_meta_path) {
        auto r = resld.lookup_resource(pack_meta_path);
        if (!r) {
            return asset_pack({}, {}, {}, {}, {}, {});
        }

        util::resource &meta_res = *r;

        if (!meta_res("beginning"_f) || meta_res("size"_f) == 0) {
            return asset_pack({}, {}, {}, {}, {}, {});
        }

        std::string content(reinterpret_cast<const char*>(meta_res("beginning"_f)),
        ↪ meta_res("size"_f));
        auto keys = parse_meta_keys(content);

        std::vector<texture> walls = keys.count("wall") ? tx_from_meta(resld,
        ↪ keys.at("wall")) : std::vector<texture>{};
        std::vector<texture> sprites = keys.count("sprite") ? tx_from_meta(resld,
        ↪ keys.at("sprite")) : std::vector<texture>{};
        std::vector<texture> flats = keys.count("flat") ? tx_from_meta(resld,
        ↪ keys.at("flat")) : std::vector<texture>{};
        std::vector<texture> ui = keys.count("ui") ? tx_from_meta(resld,
        ↪ keys.at("ui")) : std::vector<texture>{};
        std::vector<menu> menus = keys.count("menus") ? menus_from_meta(resld,
        ↪ keys.at("menus")) : std::vector<menu>{};
        std::vector<audio_clip> clips = keys.count("audio") ? clips_from_meta(resld,
        ↪ keys.at("audio")) : std::vector<audio_clip>{};

        return asset_pack(std::move(walls), std::move(sprites), std::move(flats),
        ↪ std::move(ui), std::move(menus), std::move(clips));
    }

    int asset_pack::display_menu(menu_id const id, rendering::renderer_2d const& r2d,
    ↪ rendering::rendering_backend &rb, input::input_backend &in) {
        return menu_by_id(id).display(r2d, rb, in, ui_textures);
    }
}

```

assets/audio-clip.cpp

```
#include "audio-clip.h"
#include <iostream>

namespace assets {

    audio_clip::audio_clip(std::vector<std::uint8_t> const& data, audio::audio_format
↪ const& fmt) :
        pcm_data(std::move(data)),
        format(fmt) {
            std::uint32_t bytes_per_frame = (fmt.bits_per_sample / 8) * fmt.channels;
            total_frames = (bytes_per_frame > 0) ? (pcm_data.size() / bytes_per_frame) :
↪ 0;
        }

    audio_clip const audio_clip::load_from_bin(util::resource const &res) {
        if (res("size"_f) < 12) {
            std::cerr << "empty audio clip" << std::endl;
            return audio_clip({}, {0, 0, 0});
        }

        const std::uint32_t* header = reinterpret_cast<const
↪ std::uint32_t*>(res("beginning"_f));
        audio::audio_format fmt;
        fmt.sample_rate = header[0];
        fmt.channels = header[1];
        fmt.bits_per_sample = header[2];

        const std::uint8_t* byte_data = reinterpret_cast<const
↪ std::uint8_t*>(res("beginning"_f)) + 12;
        std::size_t data_size = res("size"_f) - 12;

        std::vector<std::uint8_t> pcm(byte_data, byte_data + data_size);

        return audio_clip(pcm, fmt);
    }
}
```


assets/texture.cpp

```
#include "texture.h"

#include <iostream>

namespace assets {

    texture::texture(std::vector<std::uint32_t> const &t, std::uint32_t const w,
        ↪ std::uint32_t const h, bool const transparent)
        : pixels(std::move(t)), width(w), height(h), has_transparency(transparent) {}

    texture const texture::load_from_bin(util::resource const &res) {
        if (res("size"_f) < 12) {
            std::cerr << "empty texture" << std::endl;
            /* causes crash when used, INTENTIONAL */
            return texture({}, 0, 0, false);
        }

        const std::uint32_t* data = reinterpret_cast<const
            ↪ std::uint32_t*>(res("beginning"_f));
        std::uint32_t w = data[0];
        std::uint32_t h = data[1];
        bool has_transparency = data[2] != 0;

        const std::uint8_t* byte_data = reinterpret_cast<const
            ↪ std::uint8_t*>(res("beginning"_f)) + 12;

        std::vector<std::uint32_t> pixels;
        pixels.reserve(w * h);

        for (std::uint32_t i = 0; i < w * h; ++i) {
            std::uint32_t r = byte_data[i * 4 + 0];
            std::uint32_t g = byte_data[i * 4 + 1];
            std::uint32_t b = byte_data[i * 4 + 2];
            std::uint32_t a = byte_data[i * 4 + 3];

            std::uint32_t pixel_color = (a << 24) | (r << 16) | (g << 8) | b;
            pixels.push_back(pixel_color);
        }

        return texture(pixels, w, h, has_transparency);
    }
}
```

assets/menu-element.cpp

```
#include "menu-element.h"
#include "rendering/renderer-2d.h"
#include "input/input-backend.h"

#include <sstream>
#include <limits>

namespace assets {

    menu_element::menu_element(int f, float bx, float by, float ex, float ey,
    ↪ texture_id const b, std::string s, int cw, int ch, std::uint8_t r, std::uint8_t g,
    ↪ std::uint8_t b_col, float rlm) :
        function(f),
        begin_x(bx),
        begin_y(by),
        end_x(ex),
        end_y(ey),
        bg(b),
        fmt_template(std::move(s)),
        char_w(cw),
        char_h(ch),
        col_r(r),
        col_g(g),
        col_b(b_col),
        relative_left_margin(rlm) {}

    /* bin fmt: */
    /* <function> <bx> <by> <ex> <ey> <bg_tx_idx> <char_w> <char_h> <r> <g> <b>
    ↪ <relative_left_margin> */
    /* <format template string - rest of next line> */
    menu_element menu_element::load_from_bin(util::resource const &res) {
        std::string content(reinterpret_cast<const char *>(res("beginning"_f)),
        ↪ res("size"_f));
        std::istringstream stream(content);

        int f;
        float bx, by, ex, ey;
        int bg_tx_idx;
        int cw, ch;
        int r, g, b;
        float rlm;
        std::string tpl;

        stream >> f >> bx >> by >> ex >> ey >> bg_tx_idx
            >> cw >> ch >> r >> g >> b >> rlm;

        stream.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::getline(stream, tpl);

        return menu_element(f, bx, by, ex, ey, bg_tx_idx, std::move(tpl), cw, ch,
        ↪ static_cast<std::uint8_t>(r), static_cast<std::uint8_t>(g),
        ↪ static_cast<std::uint8_t>(b), rlm);
    }

    std::string menu_element::resolve_text() const {
        if (formatter) {
```

```

        return formatter(fmt_template);
    }
    return fmt_template;
}

std::uint32_t menu_element::text_color() const {
    return (0xFFu << 24)
        | (static_cast<std::uint32_t>(col_r) << 16)
        | (static_cast<std::uint32_t>(col_g) << 8)
        | static_cast<std::uint32_t>(col_b);
}

void menu_element::selective_fmt(std::string_view key_filter,
    ↪ std::function<std::string(std::string_view)> fn) {
    if (key_filter.empty()) {
        formatter = fn;
        return;
    }
    std::string filter(key_filter);
    std::istringstream stream(filter);
    std::string token;
    while (std::getline(stream, token, '|')) {
        if (!token.empty() && fmt_template.find(token) != std::string::npos) {
            formatter = fn;
            return;
        }
    }
}

int menu_element::render(rendering::renderer_2d const &r2d, std::vector<texture>
    ↪ const& ui_tx, int sw, int sh, input::mouse_state const &ms) const {
    int x0 = static_cast<int>(begin_x * sw);
    int y0 = static_cast<int>(begin_y * sh);
    int x1 = static_cast<int>(end_x * sw);
    int y1 = static_cast<int>(end_y * sh);
    int w = x1 - x0;
    int h = y1 - y0;

    if (bg >= 0) {
        r2d.draw_texture(ui_tx[bg], x0, y0, w, h);
    } else {
        r2d.draw_rect(x0, y0, w, h, 0x80404040);
    }

    std::string label = resolve_text();
    if (!label.empty()) {
        int text_x = x0 + static_cast<int>(relative_left_margin * w);
        int text_y = y0 + (h - char_h) / 2;
        r2d.draw_text(label, text_x, text_y, char_w, char_h, text_color());
    }

    if (ms.left && ms.x >= x0 && ms.x < x1 && ms.y >= y0 && ms.y < y1) {
        return function;
    }

    return -2;
}
}

```

assets/menu.cpp

```
#include "menu.h"
#include "rendering/renderer-2d.h"
#include "rendering/rendering-backend.h"

#include <sstream>

namespace assets {

    menu::menu(std::vector<menu_element> els, texture_id const b) :
        elements(std::move(els)),
        bg(b) {}

    /* bin fmt */
    /* <bg_tx_idx> */
    /* <element_resource_name> ... */
    menu menu::load_from_bin(util::resource_loader &resld, util::resource const &res)
    ↪ {
        std::string content(reinterpret_cast<const char *>(res("beginning"_f)),
            ↪ res("size"_f));
        std::istringstream stream(content);

        int bg_tx_idx;
        stream >> bg_tx_idx;

        std::vector<menu_element> els;
        std::string el_res_name;
        while (stream >> el_res_name) {
            util::resource const *const r = resld.lookup_resource(el_res_name);
            if (r) {
                els.push_back(menu_element::load_from_bin(*r));
            }
        }

        return menu(std::move(els), bg_tx_idx);
    }

    void menu::selective_formatter(std::string_view key_filter,
    ↪ std::function<std::string(std::string_view)> fn) {
        for (menu_element &el : elements) {
            el.selective_fmt(key_filter, fn);
        }
    }

    void menu::formatter_all(std::function<std::string(std::string_view)> fn) {
        selective_formatter("", fn);
    }

    int menu::display(rendering::renderer_2d const &r2d, rendering::rendering_backend
    ↪ &rb, input::input_backend &in, std::vector<texture> const& ui_tx) const {
        const int sw = static_cast<int>(rb("width"_f));
        const int sh = static_cast<int>(rb("height"_f));

        in.reset_mouse_state(sw / 2, sh / 2);

        bool esc_debounce = in.is_key_down(input::key::esc);
    }
```

```

while (true) {
    in.update();

    if (bg >= 0) {
        r2d.draw_texture(ui_tx[bg], 0, 0, sw, sh);
    } else {
        r2d.draw_rect(0, 0, sw, sh, 0xFF000000);
    }

    input::mouse_state const ms = in.get_mouse_state();

    for (menu_element const &el : elements) {
        int result = el.render(r2d, ui_tx, sw, sh, ms);
        if (result != -2) {
            return result;
        }
    }

    const int cw = static_cast<int>(ui_tx[1]("width"_f));
    const int ch = static_cast<int>(ui_tx[1]("height"_f));
    r2d.draw_texture(ui_tx[1], ms.x, ms.y, cw, ch);

    if (esc_debounce) {
        if (!in.is_key_down(input::key::esc)) esc_debounce = false;
    } else if (in.is_key_down(input::key::esc)) {
        return -1;
    }

    rb.flush();
}
}
}

```

audio/alsa/backend.cpp

```
#include "backend.h"

namespace audio {
    namespace alsa {

        backend::backend() :
            is_bad(false),
            pcm_handle(nullptr),
            fmt{48000, 2, 16},
            paused(false) {}

        backend::~backend() {
            close();
        }

        snd_pcm_format_t backend::to_alsa_format(unsigned int bits) const {
            switch (bits) {
                case 8: return SND_PCM_FORMAT_U8;
                case 16: return SND_PCM_FORMAT_S16_LE;
                case 24: return SND_PCM_FORMAT_S24_LE;
                case 32: return SND_PCM_FORMAT_S32_LE;
                default: return SND_PCM_FORMAT_S16_LE;
            }
        }

        bool backend::configure_hw_params() {
            snd_pcm_hw_params_t* params;
            snd_pcm_hw_params_alloca(&params);

            if (snd_pcm_hw_params_any(pcm_handle, params) < 0)
                return false;

            if (snd_pcm_hw_params_set_access(pcm_handle, params,
                ↪ SND_PCM_ACCESS_RW_INTERLEAVED) < 0)
                return false;

            if (snd_pcm_hw_params_set_format(pcm_handle, params,
                ↪ to_alsa_format(fmt.bits_per_sample)) < 0)
                return false;

            if (snd_pcm_hw_params_set_channels(pcm_handle, params, fmt.channels) < 0)
                return false;

            unsigned int rate = fmt.sample_rate;
            if (snd_pcm_hw_params_set_rate_near(pcm_handle, params, &rate, nullptr) <
                ↪ 0)
                return false;
            fmt.sample_rate = rate;

            snd_pcm_uframes_t buffer_size = rate / 10;
            snd_pcm_uframes_t period_size = rate / 40;
            snd_pcm_hw_params_set_buffer_size_near(pcm_handle, params, &buffer_size);
            snd_pcm_hw_params_set_period_size_near(pcm_handle, params, &period_size,
                ↪ nullptr);

            if (snd_pcm_hw_params(pcm_handle, params) < 0)
```

```

        return false;

    return true;
}

bool backend::open(audio_format const& format) {
    close();
    fmt = format;

    int err = snd_pcm_open(&pcm_handle, "pulse", SND_PCM_STREAM_PLAYBACK, 0);
    if (err < 0)
        err = snd_pcm_open(&pcm_handle, "pipewire", SND_PCM_STREAM_PLAYBACK,
↪ 0);
    if (err < 0)
        err = snd_pcm_open(&pcm_handle, "default", SND_PCM_STREAM_PLAYBACK,
↪ 0);
    if (err < 0) {
        is_bad = true;
        pcm_handle = nullptr;
        return false;
    }

    if (!configure_hw_params()) {
        snd_pcm_close(pcm_handle);
        pcm_handle = nullptr;
        is_bad = true;
        return false;
    }

    if (snd_pcm_prepare(pcm_handle) < 0) {
        snd_pcm_close(pcm_handle);
        pcm_handle = nullptr;
        is_bad = true;
        return false;
    }

    is_bad = false;
    paused = false;
    return true;
}

void backend::drain() {
    if (pcm_handle && !is_bad)
        snd_pcm_drain(pcm_handle);
}

void backend::close() {
    if (pcm_handle) {
        snd_pcm_drain(pcm_handle);
        snd_pcm_close(pcm_handle);
        pcm_handle = nullptr;
    }
    paused = false;
}

long backend::write(void const* data, unsigned long frames) {
    if (!pcm_handle || is_bad || paused) return -1;

    snd_pcm_sframes_t written = snd_pcm_writei(

```

```

        pcm_handle,
        data,
        static_cast<snd_pcm_uframes_t>(frames)
    );

    if (written == -EPIPE) {
        snd_pcm_prepare(pcm_handle);
        written = snd_pcm_writei(
            pcm_handle,
            data,
            static_cast<snd_pcm_uframes_t>(frames)
        );
    }

    if (written < 0) {
        is_bad = true;
        return -1;
    }

    return static_cast<long>(written);
}

void backend::pause() {
    if (!pcm_handle || is_bad) return;

    if (snd_pcm_pause(pcm_handle, 1) < 0)
        snd_pcm_drop(pcm_handle);

    paused = true;
}

void backend::resume() {
    if (!pcm_handle || is_bad) return;

    if (snd_pcm_pause(pcm_handle, 0) < 0)
        snd_pcm_prepare(pcm_handle);

    paused = false;
}

audio_format backend::operator()(util::component_tag<"current_format">) const
→ {
    return fmt;
}

bool backend::bad() const {
    return is_bad;
}
}
}

```


audio/audio-mixer.cpp

```
#include "audio-mixer.h"
#include <algorithm>
#include <iostream>

namespace audio {

    audio_mixer::audio_mixer(audio_backend &tgt, audio_format const& fmt)
        : target(tgt), mix_format(fmt) {
        target.open(mix_format);
    }

    void audio_mixer::play(assets::audio_clip const& clip) {
        audio_format const clip_fmt = clip("format"_f);

        if (clip_fmt.sample_rate != mix_format.sample_rate ||
            clip_fmt.bits_per_sample != 16) {
            std::cerr << "Warning: Clip format doesn't match mixer format!" <<
                << std::endl;
            return;
        }

        active_sounds.push_back({&clip, 0});
    }

    void audio_mixer::stop_all() {
        active_sounds.clear();
    }

    void audio_mixer::step(unsigned long frames_to_mix) {
        if (target.bad()) return;

        unsigned int const channels = mix_format.channels;
        unsigned long const total_samples = frames_to_mix * channels;

        std::vector<std::int32_t> mix_buffer(total_samples, 0);

        for (auto it = active_sounds.begin(); it != active_sounds.end(); ) {
            assets::audio_clip const* clip = it->clip;
            unsigned long const clip_frames = (*clip)("total_frames"_f);

            auto const* pcm_16 = reinterpret_cast<std::int16_t>
                << const*>((*clip)("pcm_data"_f).data());

            unsigned long frames_to_read = std::min(frames_to_mix, clip_frames -
                << it->current_frame);
            unsigned long samples_to_read = frames_to_read * channels;
            unsigned long start_sample = it->current_frame * channels;

            for (unsigned long i = 0; i < samples_to_read; ++i) {
                mix_buffer[i] += pcm_16[start_sample + i];
            }

            it->current_frame += frames_to_read;

            if (it->current_frame >= clip_frames) {
                it = active_sounds.erase(it);
            }
        }
    }
}
```

```

        } else {
            ++it;
        }
    }

    std::vector<std::int16_t> final_buffer(total_samples);
    for (unsigned long i = 0; i < total_samples; ++i) {
        final_buffer[i] = static_cast<std::int16_t>(
            std::clamp(mix_buffer[i], -32768, 32767)
        );
    }

    target.write(final_buffer.data(), frames_to_mix);
}

bool audio_mixer::is_silent() const {
    return active_sounds.empty();
}
}

```

game/game.cpp

```
#include "game.h"

#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <cstring>
#include <cmath>

#include "rendering/drm-kms/backend.h"
#include "audio/alsa/backend.h"
#include "input/evdev/backend.h"
#include "entities/monster-factory.h"
#include "entities/monsters/monster-boss.h"
#include "world_objects/pickup-factory.h"
#include "combat/weapons/pistol.h"

namespace game {

    game::game() :
        rl{},
        am(assets::asset_manager::load(rl)),
        input(std::make_unique<input::evdev::backend>()),
        rb(std::make_unique<rendering::drm_kms::backend>()),
        ab(std::make_unique<audio::alsa::backend>()),
        md(geometry::map_data::load_from_bin(
            *rl.lookup_resource("map/sectors.bin"),
            *rl.lookup_resource("map/sidedefs.bin"),
            *rl.lookup_resource("map/linedefs.bin"),
            *rl.lookup_resource("map/subsectors.bin"),
            *rl.lookup_resource("map/nodes.bin"),
            *rl.lookup_resource("map/monsters.bin"),
            *rl.lookup_resource("map/pickups.bin"))),
        mix(*ab, audio::audio_format{48000, 2, 16}),
        r2d(*rb, am, am.ui_tx_by_id(0)),
        sr(*rb, am, md),
        w(),
        player_ptr(nullptr),

        fov(1.55) {
        if (input->is_bad()) {
            std::cerr << "error: failed to initialize input backend\n";
            std::exit(1);
        }

        if (rb->bad()) {
            std::cerr << "error: failed to initialize rendering backend\n";
            std::exit(1);
        }

        auto modes = (*rb)("modes"_f);
        if (modes.empty()) {
            std::cerr << "error: no rendering modes available\n";
            std::exit(1);
        }
    }
```

```

    }
    rb->push_mode(std::move(modes[0]));

    if (rb->bad()) {
        std::cerr << "error: rendering backend entered bad state after mode
        ↪ set\n";
        std::exit(1);
    }

    if (ab->bad()) {
        std::cerr << "error: failed to initialize audio backend\n";
        std::exit(1);
    }
}

void game::text_scroll(std::vector<std::string> const& text, assets::audio_clip_id
    ↪ const aid) {
    int const screen_w = (*rb)("width"_f);
    int const screen_h = (*rb)("height"_f);

    int const horizon_y = screen_h / 8;
    int const base_char_w = 32;
    int const base_char_h = 48;
    int const line_spacing = 60;

    float const total_scroll = screen_h + static_cast<float>(text.size() *
    ↪ line_spacing) + horizon_y;
    float scroll_offset = 0.0f;

    mix.play(am.audio_clip_by_id(aid));

    auto last_tick = std::chrono::high_resolution_clock::now();

    while (scroll_offset < total_scroll) {
        float last_line_flat_y = screen_h - scroll_offset +
        ↪ static_cast<float>((text.size() - 1) * line_spacing);
        if (last_line_flat_y <= horizon_y) break;

        std::memset(const_cast<std::uint32_t *>((*rb)("mmio"_f)), 0x00, screen_h *
        ↪ (*rb)("pitch"_f));

        for (std::size_t i = 0; i < text.size(); ++i) {
            float flat_y = screen_h - scroll_offset + static_cast<float>(i *
            ↪ line_spacing);

            if (flat_y <= horizon_y || flat_y > screen_h + line_spacing)
                continue;

            float time = (flat_y - horizon_y) / static_cast<float>(screen_h -
            ↪ horizon_y);
            float scale = std::pow(time, 0.6666f);
            int char_w = static_cast<int>(base_char_w * scale);
            int char_h = static_cast<int>(base_char_h * scale);

            if (char_w == 0 || char_h == 0) continue;

            int draw_x = (screen_w - static_cast<int>(text[i].length()) * char_w)
            ↪ / 2;
            int draw_y = static_cast<int>(flat_y);

```

```

        r2d.draw_text(text[i].c_str(), draw_x, draw_y, char_w, char_h,
↪ 0x007fff);
    }

    rb->flush();

    auto now = std::chrono::high_resolution_clock::now();
    float dt_secs = std::chrono::duration<float>(now - last_tick).count();
    last_tick = now;

    mix.step(static_cast<int>(48000 * dt_secs));
    rb->wait_for_vsync();

    scroll_offset += 90 * dt_secs;
}

mix.stop_all();
}

void game::ts_from_resource(std::string const& res_name, assets::audio_clip_id
↪ const aid) {
    util::resource const* res = rl.lookup_resource(res_name);
    if (!res) {
        std::cerr << "error: resource not found: " << res_name << "\n";
        return;
    }

    std::string_view content(static_cast<char const*>((*res)("beginning"_f)),
↪ (*res)("size"_f));

    std::vector<std::string> lines;
    std::string_view::size_type pos = 0;
    while (pos < content.size()) {
        auto nl = content.find('\n', pos);
        if (nl == std::string_view::npos) nl = content.size();
        lines.emplace_back(content.substr(pos, nl - pos));
        pos = nl + 1;
    }

    text_scroll(lines, aid);
}

void game::credits() {
    ts_from_resource("credits", 0);
}

void game::opening() {
    ts_from_resource("opening-scroll", 14);
}

void game::ending() {
    ts_from_resource("ending-scroll", 0);
}

void game::show_main_menu() {
    while(1) {
        int result = am.display_menu(0, r2d, *rb, *input);
    }
}

```

```

        switch (result) {
            case -1:
                return;
            case 0:
                loop();
                break;
            case 1:
                credits();
                break;
            case 2:
                show_options();
                break;
            default:
                break;
        }
    }
}

void game::show_options() {
    auto modes = (*rb)("modes"_f);
    std::size_t mode_idx = 0;

    while(1) {
        assets::menu& menu = am.menu_by_id(1);

        menu.selective_formatter(
            "{mode}|{fov}",
            [&](std::string_view tpl) {
                struct replacement { std::string_view marker; std::string value;
                ↵ };
                replacement replacements[] = {
                    { "{mode}", modes.empty() ? "N/A" :
                        std::to_string((*modes[mode_idx])("x_res"_f))+"x"+std::
                        ↵ to_string((*modes[mode_idx])("y_res"_f))+"
                        ↵ "+std::to_string((*modes[mode_idx])("refresh_hz"_f))+
                        ↵ "Hz" },
                    { "{fov}", std::to_string(fov) },
                };

                std::string result(tpl);
                for (auto const& r : replacements) {
                    for (std::size_t pos = 0;
                        (pos = result.find(r.marker, pos)) != std::string::npos;)
                        ↵ {
                            result.replace(pos, r.marker.size(), r.value);
                            pos += r.value.size();
                        }
                }
                return result;
            }
        );

        int result = am.display_menu(1, r2d, *rb, *input);

        switch (result) {
            case -1:
                return;
            case 0:
                am.cycle_set();

```

```

        break;
    case 1:
        if (!modes.empty()) {
            mode_idx = (mode_idx + modes.size() - 1) % modes.size();
            rb->push_mode(std::move(modes[mode_idx]));
            modes = (*rb)("modes"_f);
        }
        break;
    case 2:
        if (!modes.empty()) {
            mode_idx = (mode_idx + 1) % modes.size();
            rb->push_mode(std::move(modes[mode_idx]));
            modes = (*rb)("modes"_f);
        }
        break;
    case 3:
        if (fov > 0.5) fov -= 0.05;
        break;
    case 4:
        if (fov < 3) fov += 0.05;
        break;
    default:
        break;
}
}
}

void game::loop() {
    constexpr float cam_height = 48.0f;
    constexpr float mouse_sens = 0.002f;

    auto p = std::make_unique<entities::player>(
        math::vec2{0.0f, 0.0f}, 0.0f, 0, 1.0f,
        100.0f, 50.0f, 120.0f, 1.0f, &md, &w);
    player_ptr = &*p;

    player_ptr->equip(0, std::make_unique<combat::weapons::pistol>(md, w, mix,
↪ am));

    w.register_entity(std::move(p));

    bool has_bsp = md.root_node_id !=
↪ util::indexed_storage<geometry::bsp_node>::nullid;

    std::vector<entities::monster*> alive_monsters;
    entities::monster_boss* boss_ptr = nullptr;

    for (auto const& spawn : md.monster_spawns) {
        auto m = entities::make_monster(spawn, *player_ptr, md, w);
        if (m) {
            auto* raw = &*m;
            if (auto* boss = dynamic_cast<entities::monster_boss*>(raw))
                boss_ptr = boss;
            auto sub_id = has_bsp ? md.get_subsector_id(spawn("pos"_f)) : 1;
            w.register_entity(std::move(m));
            md.subsectors[sub_id].add_sprite(raw);
            alive_monsters.push_back(raw);
        }
    }
}

```

```

for (auto const& spawn : md.pickup_spawns) {
    auto sub_id = has_bsp ? md.get_subsector_id(spawn("pos"_f)) : 1;
    auto pk = world_object::make_pickup(spawn, *player_ptr, md, sub_id, w,
        ↪ mix, am);
    if (pk) {
        auto* raw = &*pk;
        w.register_entity(std::move(pk));
        md.subsectors[sub_id].add_sprite(raw);
    }
}

int prev_mouse_x = 0;
bool prev_r = false;
bool boss_dead = false;
float boss_dead_timer = 0.0f;
float damage_flash = 0.0f;
float prev_hp = (*player_ptr)("health"_f)("current_hp"_f);
auto last_tick = std::chrono::high_resolution_clock::now();

while(1) {
    auto now = std::chrono::high_resolution_clock::now();
    float dt = std::chrono::duration<float>(now - last_tick).count();
    last_tick = now;

    if (dt > 0.5f) dt = 0.5f;

    input->update();
    if (input->is_key_down(input::key::esc)) break;

    // movement
    float fwd = 0.0f, strafe = 0.0f;
    if (input->is_key_down(input::key::w)) strafe -= dt;
    if (input->is_key_down(input::key::s)) strafe += dt;
    if (input->is_key_down(input::key::a)) fwd -= dt;
    if (input->is_key_down(input::key::d)) fwd += dt;
    if (input->is_key_down(input::key::v)) player_ptr->start_sprint();
    if (input->is_key_down(input::key::c)) player_ptr->stop_sprint();
    if (fwd != 0.0f || strafe != 0.0f)
        player_ptr->move({strafe, fwd});

    auto mouse = input->get_mouse_state();
    int delta_x = mouse.x - prev_mouse_x;
    prev_mouse_x = mouse.x;
    if (delta_x != 0)
        player_ptr->rotate(static_cast<float>(-delta_x) * mouse_sens);

    if (input->is_key_down(input::key::n1)) player_ptr->switch_weapons(0);
    if (input->is_key_down(input::key::n2)) player_ptr->switch_weapons(1);
    if (input->is_key_down(input::key::n3)) player_ptr->switch_weapons(2);
    if (input->is_key_down(input::key::n4)) player_ptr->switch_weapons(3);
    if (input->is_key_down(input::key::n5)) player_ptr->switch_weapons(4);
    if (input->is_key_down(input::key::n6)) player_ptr->switch_weapons(5);
    if (input->is_key_down(input::key::n7)) player_ptr->switch_weapons(6);

    if (mouse.left)
        player_ptr->shoot();
    bool cur_r = input->is_key_down(input::key::r);
    if (cur_r && !prev_r)

```



```

        player_ptr->reload();
    prev_r = cur_r;

    w.update(dt);

    float cur_hp = (*player_ptr)("health"_f)("current_hp"_f);
    if (cur_hp < prev_hp)
        damage_flash = 0.25f;
    prev_hp = cur_hp;
    if (damage_flash > 0.0f)
        damage_flash -= dt;

    for (auto it = alive_monsters.begin(); it != alive_monsters.end(); ) {
        if ((*it)->is_dead()) {
            auto* spr = static_cast<rendering::sprite*>(*it);
            auto sub_id = has_bsp ? md.get_subsector_id((*spr)("pos"_f)) : 1;
            md.subsectors[sub_id].remove_sprite(spr);
            it = alive_monsters.erase(it);
        } else {
            ++it;
        }
    }

    if (boss_ptr && boss_ptr->is_dead() && !boss_dead) {
        boss_dead = true;
        boss_dead_timer = 10.0f;
    }
    if (boss_dead) {
        boss_dead_timer -= dt;
        if (boss_dead_timer <= 0.0f) {
            ending();
            break;
        }
    }
}

// player death → game over screen
if (player_ptr->is_dead()) {
    int const sw = (*rb)("width"_f);
    int const sh = (*rb)("height"_f);
    std::memset(const_cast<std::uint32_t*>((*rb)("mmio"_f)), 0x00,
        static_cast<std::size_t>(sh) * (*rb)("pitch"_f));
    int const cw = 48;
    int const ch = 72;
    std::string_view text = "GAME OVER";
    int tx = sw / 2 - static_cast<int>(text.size()) * cw / 2;
    int ty = sh / 2 - ch / 2;
    r2d.draw_text(text, tx, ty, cw, ch, 0xFF0000);
    rb->flush();

    while (true) {
        input->update();
        if (input->is_key_down(input::key::esc) ||
            input->is_key_down(input::key::enter) ||
            input->is_key_down(input::key::space))
            break;
        rb->wait_for_vsync();
    }
    break;
}

```

```

    auto& spr =
        ↪ static_cast<util::componentized<rendering::sprite>&>(*player_ptr);

    sr.render_bsp(spr("pos"_f), cam_height, spr("angle"_f), fov);

    draw_hud();

    if (damage_flash > 0.0f) {
        int const sw = (*rb)("width"_f);
        int const sh = (*rb)("height"_f);
        std::uint32_t alpha = static_cast<std::uint32_t>(
            (damage_flash / 0.25f) * 100.0f);
        r2d.draw_rect(0, 0, sw, sh, (alpha << 24) | 0xFF0000);
    }

    rb->flush();

    mix.step(static_cast<int>(48000.0f * dt));

    rb->wait_for_vsync();
}

}

void game::draw_hud() {
    if (!player_ptr) return;

    int const sw = (*rb)("width"_f);
    int const sh = (*rb)("height"_f);

    // HUD scale
    constexpr int hud_scale = 4;

    // Crosshair
    auto const& ch_tex = am.ui_tx_by_id(9);
    int const ch_size = 32 * 2;
    r2d.draw_texture(ch_tex, sw / 2 - ch_size / 2, sh / 2 - ch_size / 2, ch_size,
        ↪ ch_size);

    // Reload bar
    auto const& p_weapons = (*player_ptr)("weapons"_f);
    int const p_wpn_idx = (*player_ptr)("current_weapon_index"_f);
    if (p_wpn_idx >= 0 && p_wpn_idx < static_cast<int>(p_weapons.size())) {
        auto const& wpn = p_weapons[p_wpn_idx];
        if (wpn && (*wpn)("reloading"_f)) {
            int const bar_w = 80;
            int const bar_h = 8;
            int const bar_x = sw / 2 - bar_w / 2;
            int const bar_y = sh / 2 + ch_size / 2 + 8;
            float frac = (*wpn)("reload_timer"_f) / (*wpn)("reload_duration"_f);
            if (frac > 1.0f) frac = 1.0f;
            r2d.draw_rect(bar_x, bar_y, bar_w, bar_h, 0x80333333);
            r2d.draw_rect(bar_x, bar_y, static_cast<int>(bar_w * frac), bar_h,
        ↪ 0xFFFFFFFF);
        }
    }

    // HP HUD
    auto const& hp_tex = am.ui_tx_by_id(6);

```

```

int const hp_w = hp_tex("width"_f) * hud_scale;
int const hp_h = hp_tex("height"_f) * hud_scale;
int const hp_x = 0;
int const hp_y = sh - hp_h;

r2d.draw_texture(hp_tex, hp_x, hp_y, hp_w, hp_h);

// HP bar
auto const& p_health = (*player_ptr)("health"_f);
float hp_frac = p_health("current_hp"_f) / p_health("max_hp"_f);
if (hp_frac < 0.0f) hp_frac = 0.0f;
if (hp_frac > 1.0f) hp_frac = 1.0f;
int const bar_max_w = 46 * hud_scale;
int const bar_h = 10 * hud_scale;
int const hp_bar_x = hp_x + 18 * hud_scale;
int const hp_bar_y = hp_y + 3 * hud_scale;
r2d.draw_rect(hp_bar_x, hp_bar_y, static_cast<int>(bar_max_w * hp_frac),
↪ bar_h, 0xFFFF0000);

// Armor bar
float arm_frac = 0.0f;
if (p_health("max_armor"_f) > 0.0f)
    arm_frac = p_health("armor"_f) / p_health("max_armor"_f);
if (arm_frac < 0.0f) arm_frac = 0.0f;
if (arm_frac > 1.0f) arm_frac = 1.0f;
int const arm_bar_x = hp_x + 18 * hud_scale;
int const arm_bar_y = hp_y + 19 * hud_scale;
r2d.draw_rect(arm_bar_x, arm_bar_y, static_cast<int>(bar_max_w * arm_frac),
↪ bar_h, 0xFF4488FF);

// Mag HUD
auto const& mag_tex = am.ui_tx_by_id(7);
int const mag_w = mag_tex("width"_f) * hud_scale;
int const mag_h = mag_tex("height"_f) * hud_scale;
int const mag_x = sw - mag_w;
int const mag_y = sh - mag_h;

r2d.draw_texture(mag_tex, mag_x, mag_y, mag_w, mag_h);

// Ammo and reserve mags
if (p_wpn_idx >= 0 && p_wpn_idx < static_cast<int>(p_weapons.size())) {
    auto const& wpn = p_weapons[p_wpn_idx];
    if (wpn) {
        std::string ammo_str = std::to_string((*wpn)("ammo_count"_f));
        std::string mags_str = std::to_string((*wpn)("reserve_mags"_f));
        int const ammo_cw = 20;
        int const ammo_ch = 28;
        int ammo_tx = mag_x + mag_w / 2 - static_cast<int>(ammo_str.size()) *
↪ ammo_cw / 2;
        int ammo_ty = mag_y + 2 * hud_scale;
        r2d.draw_text(ammo_str, ammo_tx, ammo_ty, ammo_cw, ammo_ch, 0xFFFFFFFF);
        int const mag_cw = 14;
        int const mag_ch = 20;
        int mags_tx = mag_x + mag_w / 2 - static_cast<int>(mags_str.size()) *
↪ mag_cw / 2;
        int mags_ty = mag_y + mag_h - mag_ch - 3 * hud_scale;
        r2d.draw_text(mags_str, mags_tx, mags_ty, mag_cw, mag_ch, 0xCCCCCC);
    }
}
}

```

```

    // Weapon inventory
    auto const& inv_tex = am.ui_tx_by_id(8);
    int const inv_w = inv_tex("width"_f) * hud_scale;
    int const inv_h = inv_tex("height"_f) * hud_scale;
    int const inv_x = mag_x - inv_w;
    int const inv_y = sh - inv_h;

    r2d.draw_texture(inv_tex, inv_x, inv_y, inv_w, inv_h);

    // Draw weapon sprites (almost) next to their slot numbers
    int const cell_w = 32 * hud_scale;
    int const cell_h = 8 * hud_scale;
    int const sprite_size = 6 * hud_scale;

    for (int i = 0; i < static_cast<int>(p_weapons.size()) && i < 7; ++i) {
        if (!p_weapons[i]) continue;
        int col = i / 4;
        int row = i % 4;
        int slot_x = inv_x + col * cell_w + 10 * hud_scale;
        int slot_y = inv_y + row * cell_h;
        auto const& weapon_sprite = am.sprite_tx_by_id(15 + i);
        r2d.draw_texture(weapon_sprite, slot_x, slot_y + 1 * hud_scale,
↪ sprite_size, sprite_size);

        if (i == p_wpn_idx) {
            r2d.draw_rect(slot_x - 1, slot_y, sprite_size + 2, cell_h,
↪ 0x80FFFF00);
        }
    }

    void game::run() {
        //opening();
        show_main_menu();
    }
}

```

main.cpp

```
#include "game/game.h"

int main() {
    game::game g;
    g.run();
    return 0;
}
```

entities/monsters/monster-all-rounder.cpp

```
#include "monster-all-rounder.h"

namespace entities {

monster_all_rounder::monster_all_rounder(math::vec2 const p, float const z,
↪ engine::actor& tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 0, 1.0f, 80.0f, 20.0f, 80.0f, 100.0f, 250.0f, 10.0f, 1.0f, tgt,
↪ map, wrld),
  melee_mode(false), melee_threshold(60.0f) {}

void monster_all_rounder::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  melee_mode = (dist < melee_threshold);

  if (melee_mode) {
    if (dist > attack_range)
      move_toward_target(movement_speed, dt);
    if (dist <= attack_range && attack_cooldown <= 0.0f) {
      melee_attack(attack_damage * 1.5f);
      attack_cooldown = attack_cd_max;
    }
  } else {
    float preferred = attack_range * 0.7f;
    if (dist < preferred - 1.0f) move_away_from_target(movement_speed, dt);
    else if (dist > preferred + 1.0f) move_toward_target(movement_speed, dt);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
      ranged_attack(attack_damage, 30, 250.0f);
      attack_cooldown = attack_cd_max;
    }
  }
}

}
```

entities/monsters/monster-assault.cpp

```
#include "monster-assault.h"

namespace entities {

monster_assault::monster_assault(math::vec2 const p, float const z, engine::actor&
↪ tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 1, 1.0f, 60.0f, 10.0f, 110.0f, 150.0f, 300.0f, 7.0f, 0.2f, tgt,
↪ map, wrld),
  burst_size(3), burst_interval(0.2f),
  burst_remaining(0), burst_timer(0.0f), burst_cooldown(0.0f),
  strafe_sign(1.0f), strafe_timer(0.0f) {}

void monster_assault::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  strafe_timer -= dt;
  if (strafe_timer <= 0.0f) {
    strafe_sign = -strafe_sign;
    strafe_timer = 1.2f;
  }
  strafe(movement_speed * 0.7f * strafe_sign, dt);

  float preferred = attack_range * 0.75f;
  if (dist > preferred + 1.5f) move_toward_target(movement_speed * 0.5f, dt);
  else if (dist < preferred - 1.5f) move_away_from_target(movement_speed * 0.5f,
↪ dt);

  if (burst_cooldown > 0.0f) { burst_cooldown -= dt; return; }

  if (burst_remaining > 0) {
    burst_timer -= dt;
    if (burst_timer <= 0.0f && dist <= attack_range) {
      ranged_attack(attack_damage, 30, 300.0f);
      --burst_remaining;
      burst_timer = burst_interval;
    }
    if (burst_remaining == 0)
      burst_cooldown = 2.5f;
  } else {
    burst_remaining = burst_size;
    burst_timer = 0.0f;
  }
}

}
```

entities/monsters/monster-basic.cpp

```
#include "monster-basic.h"

namespace entities {

monster_basic::monster_basic(math::vec2 const p, float const z, engine::actor& tgt,
↪ geometry::map_data& map, engine::world& wrld)
    : monster(p, z, 2, 1.0f, 30.0f, 0.0f, 100.0f, 40.0f, 200.0f, 10.0f, 1.0f, tgt,
↪ map, wrld) {}

void monster_basic::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (!has_target()) return;

    float dist = dist_to_target();
    if (dist > detection_radius) return;

    if (dist > attack_range)
        move_toward_target(movement_speed, dt);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
        melee_attack(attack_damage);
        attack_cooldown = attack_cd_max;
    }
}

}
```


entities/monsters/monster-boss.cpp

```
#include "monster-boss.h"
#include "monster-basic.h"
#include "combat/burning.h"
#include "combat/slowed.h"
#include "engine/projectile.h"
#include "engine/world.h"
#include <cmath>

namespace entities {

monster_boss::monster_boss(math::vec2 const p, float const z, engine::actor& tgt,
    ↪ geometry::map_data& map, engine::world& wrld)
    : monster(p, z, 3, 1.0f, 1000.0f, 300.0f, 120.0f, 350.0f, 500.0f, 20.0f, 1.0f,
    ↪ tgt, map, wrld),
    max_hp_val(1000.0f), boss_map_ref(map) {}

bool monster_boss::channeling() const { return is_channeling; }
float monster_boss::flash_phase() const { return flash_timer; }

float monster_boss::hp_ratio() const {
    float cur = health("current_hp"_f);
    return cur / max_hp_val;
}

void monster_boss::update_phase() {
    float ratio = hp_ratio();
    if (ratio <= 0.25f && current_phase < 3)
        current_phase = 3;
    else if (ratio <= 0.5f && current_phase < 2)
        current_phase = 2;
}

bool monster_boss::minions_alive() const {
    for (auto id : minion_ids) {
        auto* e = world_ref.entity_from_id(id);
        if (e) {
            auto* a = dynamic_cast<engine::actor*>(e);
            if (a && !a->is_dead()) return true;
        }
    }
    return false;
}

void monster_boss::ranged_attack_slow() {
    math::vec2 dir = dir_to_target();
    float base_angle = std::atan2(dir("y"_f), dir("x"_f));

    float start = base_angle - projectile_spread * (projectile_count - 1) / 2.0f;
    for (int i = 0; i < projectile_count; ++i) {
        float a = start + projectile_spread * static_cast<float>(i);
        math::vec2 shot_dir{std::cos(a), std::sin(a)};
        math::vec2 spawn_pos = pos + shot_dir * 20.0f;

        auto proj = std::make_unique<engine::projectile>(
            spawn_pos, z_pos, 30, 0.5f,
            shot_dir, 250.0f, attack_damage * 0.5f, 5.0f,
    
```

```

        engine::faction::enemy);

    auto* raw = &*proj;
    raw->world_ref = &world_ref;
    raw->map_ref = &boss_map_ref;
    raw->on_hit_effect = []() { return std::make_unique<combat::slowed>(3.0f, 30);
↪ };

    auto eid = world_ref.register_entity(std::move(proj));
    raw->self_id = eid;

    if (boss_map_ref("root_node_id"_f) !=
↪     util::indexed_storage<geometry::bsp_node>::nullid) {
        auto sub_id = boss_map_ref.get_subsector_id(spawn_pos);
        boss_map_ref("subsectors"_f)[sub_id].add_sprite(raw);
    }
}

void monster_boss::melee_attack_burn() {
    melee_attack(attack_damage);
    target.add_effect(std::make_unique<combat::burning>(3.0f, 5));
}

void monster_boss::start_charge() {
    if (!has_target()) return;
    is_charging = true;
    charge_timer = charge_duration;
    charge_dir = dir_to_target();
}

void monster_boss::update_charge(float dt) {
    math::vec2 new_pos = pos + charge_dir * (movement_speed * charge_speed_mult * dt);
    apply_wall_collision(new_pos);
    boss_map_ref.move_to(this, new_pos);
    charge_timer -= dt;

    float dist = dist_to_target();
    if (dist <= attack_range) {
        melee_attack(attack_damage * 2.0f);
        is_charging = false;
        post_charge_stun = stun_duration;
        return;
    }

    if (charge_timer <= 0.0f) {
        is_charging = false;
        post_charge_stun = stun_duration;
    }
}

void monster_boss::start_channel() {
    is_channeling = true;
    channel_timer = 0.0f;
    flash_timer = 0.0f;
    minion_ids.clear();

    for (int i = 0; i < 4; ++i) {

```

```

float angle_offset = static_cast<float>(i) * (std::numbers::pi_v<float> /
↳ 2.0f);
math::vec2 spawn_pos = pos + math::vec2{
    std::cos(angle_offset) * 40.0f,
    std::sin(angle_offset) * 40.0f
};

auto minion = std::make_unique<monster_basic>(spawn_pos, z_pos, target,
↳ boss_map_ref, world_ref);

auto* raw = &*minion;
auto id = world_ref.register_entity(std::move(minion));
minion_ids.push_back(id);

auto sub_id = boss_map_ref.get_subsector_id(spawn_pos);
boss_map_ref("subsectors"_f)[sub_id].add_sprite(raw);
}
}

void monster_boss::phase1_ai(float dt, float dist) {
    stance_timer -= dt;
    if (stance_timer <= 0.0f) {
        wants_melee = !wants_melee;
        stance_timer = 10.0f;
    }

    if (wants_melee) {
        if (dist > 60.0f)
            move_toward_target(movement_speed * 1.3f, dt);

        if (dist <= 60.0f && attack_cooldown <= 0.0f) {
            melee_attack_burn();
            attack_cooldown = attack_cd_max;
        }
    } else {
        float preferred = attack_range * 0.7f;
        if (dist > preferred + 1.5f)
            move_toward_target(movement_speed, dt);
        else if (dist < preferred - 1.5f)
            move_away_from_target(movement_speed, dt);

        ranged_cooldown -= dt;
        if (dist <= attack_range && ranged_cooldown <= 0.0f) {
            ranged_attack_slow();
            ranged_cooldown = ranged_cd_max;
        }
    }
}

void monster_boss::phase2_ai(float dt, float dist) {
    // charge
    if (!is_charging && charge_cooldown <= 0.0f && dist > attack_range * 0.5f) {
        start_charge();
        charge_cooldown = charge_cd_max;
        return;
    }

    // between charges, phase 1 behavior
    phase1_ai(dt, dist);
}

```

```

}

void monster_boss::phase3_ai(float dt, float dist) {
    // entering phase 3, start channel
    if (!phase3_debuffed && !is_channeling) {
        start_channel();
        return;
    }

    // channeling
    if (is_channeling) {
        flash_timer += dt * 4.0f;
        if (!minions_alive()) {
            is_channeling = false;
            phase3_debuffed = true;
            movement_speed *= 0.6f;
        }
        return;
    }

    // post-channel
    if (!is_charging && charge_cooldown <= 0.0f && dist > attack_range) {
        start_charge();
        charge_cooldown = charge_cd_max;
        return;
    }

    if (dist > attack_range)
        move_toward_target(movement_speed, dt);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
        melee_attack_burn();
        attack_cooldown = attack_cd_max;
    }
}

void monster_boss::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (!has_target()) return;

    update_phase();

    float dist = dist_to_target();
    if (dist > detection_radius) return;

    // charge in progress
    if (is_charging) {
        update_charge(dt);
        return;
    }

    // post-charge stun
    if (post_charge_stun > 0.0f) {
        post_charge_stun -= dt;
        return;
    }

    // channeling

```

```
    if (is_channeling) {  
        phase3_ai(dt, dist);  
        return;  
    }  
  
    charge_cooldown -= dt;  
  
    switch (current_phase) {  
        case 1: phase1_ai(dt, dist); break;  
        case 2: phase2_ai(dt, dist); break;  
        case 3: phase3_ai(dt, dist); break;  
    }  
}  
  
}
```

entities/monsters/monster-duzy-gruby.cpp

```
#include "monster-duzy-gruby.h"

namespace entities {

monster_Duzy_Gruby::monster_Duzy_Gruby(math::vec2 const p, float const z,
↪ engine::actor& tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 4, 1.0f, 200.0f, 50.0f, 60.0f, 45.0f, 150.0f, 25.0f, 2.0f, tgt,
↪ map, wrld) {}

void monster_Duzy_Gruby::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (!has_target()) return;

    float dist = dist_to_target();
    if (dist > detection_radius) return;

    if (dist > attack_range)
        move_toward_target(movement_speed, dt);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
        melee_attack(attack_damage);
        attack_cooldown = attack_cd_max;
    }
}

}
```

entities/monsters/monster-elite-swift.cpp

```
#include "monster-elite-swift.h"
#include "geometry/map-data.h"
#include <cmath>

namespace entities {

monster_elite_swift::monster_elite_swift(math::vec2 const p, float const z,
    ↪ engine::actor& tgt, geometry::map_data& map, engine::world& wrld)
    : monster(p, z, 5, 1.0f, 60.0f, 0.0f, 150.0f, 45.0f, 400.0f, 12.0f, 0.8f, tgt,
    ↪ map, wrld),
    charge_speed(300.0f), is_charging(false),
    charge_timer(0.0f), charge_cd(0.0f),
    circle_angle(0.0f), circle_radius(120.0f) {}

void monster_elite_swift::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (!has_target()) return;

    float dist = dist_to_target();
    if (dist > detection_radius) return;

    if (charge_cd > 0.0f) charge_cd -= dt;

    if (is_charging) {
        move_toward_target(charge_speed, dt);
        charge_timer -= dt;

        if (dist <= attack_range) {
            if (attack_cooldown <= 0.0f) {
                melee_attack(attack_damage * 2.0f);
                attack_cooldown = attack_cd_max;
            }
            is_charging = false;
            charge_cd = 5.0f;
        }
        if (charge_timer <= 0.0f) {
            is_charging = false;
            charge_cd = 5.0f;
        }
        return;
    }

    circle_angle += movement_speed * dt;
    math::vec2 new_pos{
        target("pos"_f)("x"_f) + std::cos(circle_angle) * circle_radius,
        target("pos"_f)("y"_f) + std::sin(circle_angle) * circle_radius
    };
    apply_wall_collision(new_pos);
    map_ref.move_to(this, new_pos);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
        melee_attack(attack_damage);
        attack_cooldown = attack_cd_max;
    }
}
```

```
    if (charge_cd <= 0.0f) {  
        is_charging = true;  
        charge_timer = 0.8f;  
    }  
}  
  
}
```


entities/monsters/monster-elite-tank.cpp

```
#include "monster-elite-tank.h"

namespace entities {

monster_elite_tank::monster_elite_tank(math::vec2 const p, float const z,
↪ engine::actor& tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 6, 1.0f, 400.0f, 150.0f, 60.0f, 50.0f, 200.0f, 15.0f, 1.2f, tgt,
↪ map, wrld),
  melee_mode(false), melee_threshold(70.0f),
  heavy_timer(0.0f), heavy_cd(6.0f) {}

void monster_elite_tank::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  heavy_timer -= dt;
  melee_mode = (dist < melee_threshold);

  if (melee_mode) {
    if (dist > attack_range) move_toward_target(movement_speed, dt);
    if (dist <= attack_range && attack_cooldown <= 0.0f) {
      melee_attack(attack_damage * 1.5f);
      attack_cooldown = attack_cd_max;
    }
  } else {
    float preferred = attack_range * 0.7f;
    if (dist < preferred - 1.0f) move_away_from_target(movement_speed, dt);
    else if (dist > preferred + 1.0f) move_toward_target(movement_speed, dt);

    if (dist <= attack_range && attack_cooldown <= 0.0f) {
      float dmg = (heavy_timer <= 0.0f) ? attack_damage * 3.0f : attack_damage;
      melee_attack(dmg);
      attack_cooldown = attack_cd_max;
      if (heavy_timer <= 0.0f) heavy_timer = heavy_cd;
    }
  }
}

}
```

entities/monsters/monster-magic.cpp

```
#include "monster-magic.h"

namespace entities {

monster_magic::monster_magic(math::vec2 const p, float const z, engine::actor& tgt,
↪ geometry::map_data& map, engine::world& wrld)
: monster(p, z, 7, 1.0f, 50.0f, 30.0f, 60.0f, 250.0f, 350.0f, 40.0f, 2.0f, tgt,
↪ map, wrld),
  charge_time(2.0f), charge_timer(0.0f), is_charging(false),
  post_fire_cooldown(0.0f), sidestep_timer(0.0f), sidestep_sign(1.0f) {}

void monster_magic::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  if (post_fire_cooldown > 0.0f) {
    post_fire_cooldown -= dt;
    strafe(movement_speed * sidestep_sign, dt);
    return;
  }

  if (!is_charging) {
    is_charging = true;
    charge_timer = 0.0f;
  }

  sidestep_timer -= dt;
  if (sidestep_timer <= 0.0f) {
    sidestep_sign = -sidestep_sign;
    sidestep_timer = 0.8f;
  }
  strafe(movement_speed * 0.4f * sidestep_sign, dt);

  charge_timer += dt;
  if (charge_timer >= charge_time && dist <= attack_range && attack_cooldown <=
↪ 0.0f) {
    ranged_attack(attack_damage, 30, 200.0f);
    attack_cooldown = attack_cd_max;
    is_charging = false;
    charge_timer = 0.0f;
    post_fire_cooldown = 3.0f;
  }
}

}
```

entities/monsters/monster-maly-szybki.cpp

```
#include "monster-maly-szybki.h"

namespace entities {

monster_Maly_Szybki::monster_Maly_Szybki(math::vec2 const p, float const z,
↪ engine::actor& tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 8, 1.0f, 15.0f, 0.0f, 180.0f, 40.0f, 300.0f, 12.0f, 0.5f, tgt,
↪ map, wrld),
  is_dashing(false), dash_timer(0.0f), dash_cooldown(0.0f) {}

void monster_Maly_Szybki::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  if (dash_cooldown > 0.0f) { dash_cooldown -= dt; return; }

  if (is_dashing) {
    move_toward_target(movement_speed * 3.5f, dt);
    dash_timer -= dt;

    if (dist <= attack_range) {
      if (attack_cooldown <= 0.0f) {
        melee_attack(attack_damage);
        attack_cooldown = attack_cd_max;
      }
      is_dashing = false;
      dash_cooldown = 2.5f;
    }
    if (dash_timer <= 0.0f) {
      is_dashing = false;
      dash_cooldown = 2.5f;
    }
  }
  else {
    if (dist <= detection_radius) {
      is_dashing = true;
      dash_timer = 1.0f;
    }
  }
}

}
```

entities/monsters/monster-ranged.cpp

```
#include "monster-ranged.h"

namespace entities {

monster_ranged::monster_ranged(math::vec2 const p, float const z, engine::actor& tgt,
↪ geometry::map_data& map, engine::world& wrld)
: monster(p, z, 9, 1.0f, 25.0f, 0.0f, 60.0f, 250.0f, 350.0f, 8.0f, 1.5f, tgt,
↪ map, wrld),
  preferred_dist(200.0f) {}

void monster_ranged::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  if (dist < preferred_dist - 1.0f)
    move_away_from_target(movement_speed, dt);
  else if (dist > preferred_dist + 1.0f)
    move_toward_target(movement_speed, dt);

  if (dist <= attack_range && attack_cooldown <= 0.0f) {
    ranged_attack(attack_damage, 30, 200.0f);
    attack_cooldown = attack_cd_max;
  }
}

}
```

entities/monsters/monster-sniper.cpp

```
#include "monster-sniper.h"

namespace entities {

monster_sniper::monster_sniper(math::vec2 const p, float const z, engine::actor& tgt,
↪ geometry::map_data& map, engine::world& wrld)
    : monster(p, z, 10, 1.0f, 40.0f, 0.0f, 50.0f, 500.0f, 600.0f, 30.0f, 3.0f, tgt,
↪ map, wrld),
    shoot_interval(3.0f), aim_timer(0.0f) {}

void monster_sniper::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (!has_target()) return;

    float dist = dist_to_target();
    if (dist > detection_radius) return;

    aim_timer += dt;
    if (aim_timer >= shoot_interval && dist <= attack_range && attack_cooldown <=
↪ 0.0f) {
        ranged_attack(attack_damage, 30, 350.0f);
        attack_cooldown = attack_cd_max;
        aim_timer      = 0.0f;
    }
}

}
```

entities/monsters/monster-spawner.cpp

```
#include "monster-spawner.h"

namespace entities {

monster_spawner::monster_spawner(math::vec2 const p, float const z, engine::actor&
↪  tgt, geometry::map_data& map, engine::world& wrld)
    : monster(p, z, 11, 1.0f, 100.0f, 20.0f, 0.0f, 0.0f, 350.0f, 0.0f, 0.0f, tgt,
↪  map, wrld),
    max_spawns(5), current_spawns(0),
    spawn_interval(5.0f), spawn_timer(0.0f) {}

void monster_spawner::update(float dt) {
    monster::update(dt);
    if (is_dead()) return;
    if (current_spawns >= max_spawns) return;

    spawn_timer -= dt;
    if (spawn_timer <= 0.0f) {
        ++current_spawns;
        spawn_timer = spawn_interval;
    }
}

}
```

entities/monsters/monster-trapper.cpp

```
#include "monster-trapper.h"

namespace entities {

monster_trapper::monster_trapper(math::vec2 const p, float const z, engine::actor&
↪  tgt, geometry::map_data& map, engine::world& wrld)
: monster(p, z, 12, 1.0f, 45.0f, 15.0f, 100.0f, 60.0f, 200.0f, 15.0f, 1.0f, tgt,
↪  map, wrld),
  max_traps(3), traps_placed(0),
  trap_timer(0.0f), trap_interval(2.0f),
  wander_timer(0.0f), wander_sign(1.0f) {}

void monster_trapper::update(float dt) {
  monster::update(dt);
  if (is_dead()) return;
  if (!has_target()) return;

  float dist = dist_to_target();
  if (dist > detection_radius) return;

  wander_timer -= dt;
  if (wander_timer <= 0.0f) {
    wander_sign = -wander_sign;
    wander_timer = 1.5f;
  }
  strafe(movement_speed * wander_sign, dt);
  move_toward_target(movement_speed * 0.3f, dt);

  if (traps_placed < max_traps) {
    trap_timer -= dt;
    if (trap_timer <= 0.0f) {
      ++traps_placed;
      trap_timer = trap_interval;
    }
  }
}

}
```

entities/player.cpp

```
#include "entities/player.h"
#include "geometry/map-data.h"
#include "engine/world.h"
#include <cmath>
#include <numbers>

namespace entities {

    player::player(math::vec2 const p, float const z, assets::texture_id const tex,
    ↪ float const is, float hp, float shield, float move_speed, float sens,
    ↪ geometry::map_data* map, engine::world* world)
        : engine::actor(p, z, tex, is, hp, shield, move_speed,
        ↪ engine::faction::player),
        sensitivity(sens),
        current_weapon_index(-1),
        map_ref(map),
        world_ref(world),
        walk_speed(move_speed),
        sprint_speed(move_speed * 1.5f) { weapons.resize(7); }

    void player::equip(int slot, std::unique_ptr<combat::weapons::weapon> wpn) {
        if (slot < 0 || slot >= static_cast<int>(weapons.size())) return;
        if (weapons[slot]) return;
        weapons[slot] = std::move(wpn);
        if (current_weapon_index < 0) current_weapon_index = slot;
    }

    void player::update(float dt) {
        actor::update(dt);
        for (auto& w : weapons) if (w) w->tick(dt);
    }

    void player::switch_weapons(int index) {
        if (index < 0 || index >= static_cast<int>(weapons.size())) return;
        if (!weapons[index]) return;
        current_weapon_index = index;
    }

    void player::shoot() {
        if (current_weapon_index < 0) return;
        auto& w = weapons[current_weapon_index];
        if (w && w->can_fire())
            w->fire(pos, angle);
    }

    void player::reload() {
        if (current_weapon_index < 0) return;
        auto& w = weapons[current_weapon_index];
        if (w) w->reload();
    }

    static math::vec2 closest_on_seg(math::vec2 a, math::vec2 b, math::vec2 p) {
        math::vec2 ab = b - a;
        float ab_sq = ab.sqr_len();
        if (ab_sq < 1e-8f) return a;
        float t = math::vec2::dot_product(p - a, ab) / ab_sq;
```



```

    if (t < 0.0f) t = 0.0f;
    if (t > 1.0f) t = 1.0f;
    return a + ab * t;
}

void player::move(math::vec2 direction) {
    float ca = std::cos(angle);
    float sa = std::sin(angle);
    math::vec2 move_dir{
        direction("y"_f) * ca + direction("x"_f) * sa,
        direction("y"_f) * sa - direction("x"_f) * ca
    };
    math::vec2 new_pos = pos + move_dir * movement_speed;

    if (map_ref) {
        float r = collision_radius;
        float r2 = r * r;

        // Wall collision
        for (int iter = 0; iter < 4; ++iter) {
            bool pushed = false;
            for (auto const& e : (*map_ref)("linedefs"_f)) {
                geometry::linedef const& ld = e.value;
                if (ld.is_portal()) continue;

                math::vec2 a = ld("seg"_f)("point0"_f);
                math::vec2 b = ld("seg"_f)("point1"_f);
                math::vec2 cp = closest_on_seg(a, b, new_pos);
                math::vec2 diff = new_pos - cp;
                float d2 = diff.sqr_len();

                if (d2 < r2 && d2 > 1e-8f) {
                    float d = std::sqrt(d2);
                    math::vec2 push = diff * ((r - d) / d);
                    new_pos += push;
                    pushed = true;
                }
            }
            if (!pushed) break;
        }

        // Monster collision
        if (world_ref) {
            for (auto [id, entity_ptr] : world_ref->get_entities()) {
                if (!entity_ptr) continue;
                auto* a = dynamic_cast<engine::actor*>(&*entity_ptr);
                if (!a || a == this || a->is_dead()) continue;

                math::vec2 diff = new_pos - (*a)("pos"_f);
                float d2 = diff.sqr_len();
                float min_dist = r + 16.0f; // monster radius ~16

                if (d2 < min_dist * min_dist && d2 > 1e-8f) {
                    float d = std::sqrt(d2);
                    math::vec2 push = diff * ((min_dist - d) / d);
                    new_pos += push;
                }
            }
        }
    }
}

```

```
    }

    pos = new_pos;
}

void player::start_sprint() { movement_speed = sprint_speed; }
void player::stop_sprint() { movement_speed = walk_speed; }

void player::rotate(float yaw) {
    angle += yaw * sensitivity;
    angle = std::fmod(angle, 2.0f * std::numbers::pi_v<float>);
    if (angle < 0.0f) angle += 2.0f * std::numbers::pi_v<float>;
}

}
```

entities/monster-factory.cpp

```
#include "entities/monster-factory.h"
#include "entities/monsters/monster-all-rounder.h"
#include "entities/monsters/monster-assault.h"
#include "entities/monsters/monster-basic.h"
#include "entities/monsters/monster-boss.h"
#include "entities/monsters/monster-duzy-gruby.h"
#include "entities/monsters/monster-elite-swift.h"
#include "entities/monsters/monster-elite-tank.h"
#include "entities/monsters/monster-magic.h"
#include "entities/monsters/monster-maly-szybki.h"
#include "entities/monsters/monster-ranged.h"
#include "entities/monsters/monster-sniper.h"
#include "entities/monsters/monster-spawner.h"
#include "entities/monsters/monster-trapper.h"

namespace entities {
    std::unique_ptr<monster> make_monster(geometry::monster_spawn const& ms,
        ↪ engine::actor& target, geometry::map_data& map, engine::world& world) {
        auto type = ms.type;
        auto pos = ms.pos;
        auto z = ms.z;
        switch (type) {
            case 0: return std::make_unique<monster_all_rounder>(pos, z, target,
                ↪ map, world);
            case 1: return std::make_unique<monster_assault>(pos, z, target, map,
                ↪ world);
            case 2: return std::make_unique<monster_basic>(pos, z, target, map,
                ↪ world);
            case 3: return std::make_unique<monster_boss>(pos, z, target, map,
                ↪ world);
            case 4: return std::make_unique<monster_Duzy_Gruby>(pos, z, target, map,
                ↪ world);
            case 5: return std::make_unique<monster_elite_swift>(pos, z, target,
                ↪ map, world);
            case 6: return std::make_unique<monster_elite_tank>(pos, z, target, map,
                ↪ world);
            case 7: return std::make_unique<monster_magic>(pos, z, target, map,
                ↪ world);
            case 8: return std::make_unique<monster_Maly_Szybki>(pos, z, target,
                ↪ map, world);
            case 9: return std::make_unique<monster_ranged>(pos, z, target, map,
                ↪ world);
            case 10: return std::make_unique<monster_sniper>(pos, z, target, map,
                ↪ world);
            case 11: return std::make_unique<monster_spawner>(pos, z, target, map,
                ↪ world);
            case 12: return std::make_unique<monster_trapper>(pos, z, target, map,
                ↪ world);
            default: return nullptr;
        }
    }
}
```

entities/monster.cpp

```
#include "entities/monster.h"
#include "engine/projectile.h"
#include "engine/world.h"
#include "geometry/map-data.h"
#include "geometry/linedef.h"
#include <cmath>

namespace entities {

    monster::monster(math::vec2 const p, float const z, assets::texture_id const tex,
    ↪ float const is, float hp, float shield, float move_speed, float atk_range, float
    ↪ det_radius, float atk_dmg, float atk_cd, engine::actor& tgt, geometry::map_data&
    ↪ map, engine::world& wrld)
        : engine::actor(p, z, tex, is, hp, shield, move_speed,
        ↪ engine::faction::enemy),
        target(tgt),
        map_ref(map),
        world_ref(wrld),
        attack_cooldown(0.0f),
        attack_range(atk_range),
        detection_radius(det_radius),
        attack_damage(atk_dmg),
        attack_cd_max(atk_cd) {}

    bool monster::has_target() const {
        return !target.is_dead();
    }

    float monster::dist_to_target() const {
        math::vec2 d = target("pos"_f) - pos;
        return d.len();
    }

    math::vec2 monster::dir_to_target() const {
        math::vec2 d = target("pos"_f) - pos;
        return d.normalized();
    }

    // Closest point on segment AB to point P
    static math::vec2 closest_on_seg(math::vec2 a, math::vec2 b, math::vec2 p) {
        math::vec2 ab = b - a;
        float ab_sq = ab.sqr_len();
        if (ab_sq < 1e-8f) return a;
        float t = math::vec2::dot_product(p - a, ab) / ab_sq;
        if (t < 0.0f) t = 0.0f;
        if (t > 1.0f) t = 1.0f;
        return a + ab * t;
    }

    void monster::apply_wall_collision(math::vec2& new_pos) const {
        float r = collision_radius;
        float r2 = r * r;
        for (int iter = 0; iter < 4; ++iter) {
            bool pushed = false;
            for (auto const& e : map_ref("linedefs"_f)) {
```

```

        geometry::linedef const& ld = e.value;
        if (ld.is_portal()) continue;
        math::vec2 a = ld("seg"_f)("point0"_f);
        math::vec2 b = ld("seg"_f)("point1"_f);
        math::vec2 cp = closest_on_seg(a, b, new_pos);
        math::vec2 diff = new_pos - cp;
        float d2 = diff.sqr_len();
        if (d2 < r2 && d2 > 1e-8f) {
            float d = std::sqrt(d2);
            new_pos += diff * ((r - d) / d);
            pushed = true;
        }
    }
    if (!pushed) break;
}

void monster::move_toward_target(float speed, float dt) {
    math::vec2 new_pos = pos + dir_to_target() * (speed * dt);
    apply_wall_collision(new_pos);
    map_ref.move_to(this, new_pos);
}

void monster::move_away_from_target(float speed, float dt) {
    math::vec2 new_pos = pos - dir_to_target() * (speed * dt);
    apply_wall_collision(new_pos);
    map_ref.move_to(this, new_pos);
}

void monster::strafe(float speed, float dt) {
    math::vec2 perp = dir_to_target().perpendicular();
    math::vec2 new_pos = pos + perp * (speed * dt);
    apply_wall_collision(new_pos);
    map_ref.move_to(this, new_pos);
}

void monster::melee_attack(float damage) {
    target.take_damage(damage);
}

void monster::ranged_attack(float damage, assets::texture_id tex, float speed) {
    math::vec2 dir = dir_to_target();
    math::vec2 spawn_pos = pos + dir * 20.0f;

    auto proj = std::make_unique<engine::projectile>(
        spawn_pos, z_pos, tex, 1.5f,
        dir, speed, damage, 5.0f,
        engine::faction::enemy);

    auto* raw = &*proj;
    raw->world_ref = &world_ref;
    raw->map_ref = &map_ref;

    auto eid = world_ref.register_entity(std::move(proj));
    raw->self_id = eid;

    if (map_ref("root_node_id"_f) !=
        ⇨ util::indexed_storage<geometry::bsp_node>::nullid) {
        auto sub_id = map_ref.get_subsector_id(spawn_pos);
    }
}

```

```
        map_ref("subsectors"_f)[sub_id].add_sprite(raw);
    }
}

void monster::update(float dt) {
    if (is_dead()) return;
    actor::update(dt);
    if (attack_cooldown > 0.0f) attack_cooldown -= dt;
}

}
```

systems/health_systems.cpp

```
#include "health_systems.h"
#include <algorithm>

namespace systems {

    health_system::health_system(float hp, float max, float arm, float max_arm)
        : current_hp(hp), max_hp(max), armor(arm), max_armor(max_arm) {}

    bool health_system::is_dead() const { return current_hp <= 0.0f; }

    void health_system::apply_damage(float amount) {
        if (armor > 0.0f) {
            float absorbed = std::min(armor, amount);
            armor -= absorbed;
            amount -= absorbed;
        }
        current_hp = std::max(0.0f, current_hp - amount);
    }

    void health_system::apply_heal(float amount) {
        current_hp += amount;
        if (current_hp > max_hp) current_hp = max_hp;
    }

    void health_system::apply_shield(float amount) {
        armor += amount;
        if (armor > max_armor) armor = max_armor;
    }

    void health_system::apply_true_damage(float amount) {
        current_hp = std::max(0.0f, current_hp - amount);
    }

    void health_system::add_effect(std::unique_ptr<combat::status_effect> effect,
        ↪ engine::actor& owner) {
        effect->on_apply(owner);
        active_effects.push_back(std::move(effect));
    }

    void health_system::process_effects(float dt, engine::actor& owner) {
        for (auto& effect : active_effects) {
            bool should_tick = !effect->tick(dt);
            if (should_tick) {
                effect->affect(owner);
            }
            if (effect->is_expired()) {
                effect->on_expire(owner);
            }
        }

        std::erase_if(active_effects,
            [](const auto& e) { return e->is_expired(); });
    }

}
```

world_objects/armor-pickup.cpp

```
#include "armor-pickup.h"
#include "entities/player.h"

namespace world_object {

armor_pickup::armor_pickup(math::vec2 position, float z, assets::texture_id tex,
                           float amount, entities::player& pl,
                           geometry::map_data& md,
                           util::indexed_storage<geometry::subsector>::id_t sub_id,
                           float radius)
    : pickup(position, z, tex, pl, md, sub_id, radius), armor_amount(amount) {}

void armor_pickup::on_pickup(entities::player& p) {
    p.add_shield(armor_amount);
}

}
```


world_objects/health-pickup.cpp

```
#include "health-pickup.h"
#include "entities/player.h"

namespace world_object {

health_pickup::health_pickup(math::vec2 position, float z, assets::texture_id tex,
                             float amount, entities::player& pl,
                             geometry::map_data& md,
                             util::indexed_storage<geometry::subsector>::id_t sub_id,
                             float radius)
    : pickup(position, z, tex, pl, md, sub_id, radius), heal_amount(amount) {}

void health_pickup::on_pickup(entities::player& p) {
    p.heal(heal_amount);
}

}
```

world_objects/weapon-pickup.cpp

```
#include "weapon-pickup.h"
#include "entities/player.h"
#include "combat/weapons/weapon.h"
#include <typeindex>

namespace world_object {

weapon_pickup::weapon_pickup(math::vec2 position, float z, assets::texture_id tex,
                             std::unique_ptr<combat::weapons::weapon> wpn, int
                             ↪ weapon_slot,
                             entities::player& pl, geometry::map_data& md,
                             util::indexed_storage<geometry::subsector>::id_t sub_id,
                             float radius, float scale)
: pickup(position, z, tex, pl, md, sub_id, radius, scale),
  ↪ provided_weapon(std::move(wpn)), slot(weapon_slot) {}

void weapon_pickup::on_pickup(entities::player& p) {
    if (!provided_weapon) return;
    p.equip(slot, std::move(provided_weapon));
}

}
```

world_objects/pickup-factory.cpp

```
#include "pickup-factory.h"
#include "health-pickup.h"
#include "armor-pickup.h"
#include "weapon-pickup.h"
#include "ammo-pickup.h"

#include "combat/weapons/pistol.h"
#include "combat/weapons/smg.h"
#include "combat/weapons/rifle.h"
#include "combat/weapons/shotgun.h"
#include "combat/weapons/sniper-rifle.h"
#include "combat/weapons/plasma-gun.h"
#include "combat/weapons/katana.h"

#include "geometry/map-data.h"
#include "engine/world.h"
#include "audio/audio-mixer.h"
#include "assets/asset-manager.h"

#include "geometry/pickup-spawn.h"

namespace world_object {

    // Sprite IDs (in sprite manifest):
    // 13 = health, 14 = armor
    // 15-21 = weapon pickups (pistol, smg, rifle, shotgun, sniper, plasma, katana)
    // 22-27 = ammo pickups (pistol, smg, rifle, shotgun, sniper, plasma)

    static constexpr assets::texture_id health_sprite = 13;
    static constexpr assets::texture_id armor_sprite = 14;

    static constexpr assets::texture_id weapon_sprite_base = 15;
    static constexpr assets::texture_id ammo_sprite_base = 22;

    static std::unique_ptr<pickup> make_weapon_pickup(
        std::uint32_t subtype, math::vec2 pos, float z,
        entities::player& player, geometry::map_data& md,
        util::indexed_storage<geometry::subsector>::id_t sub_id,
        engine::world& world,
        audio::audio_mixer& mixer,
        assets::asset_manager const& am)
    {
        assets::texture_id tex = weapon_sprite_base +
        ↪ static_cast<assets::texture_id>(subtype);
        std::unique_ptr<combat::weapons::weapon> w;

        switch (subtype) {
            case 0: w = std::make_unique<combat::weapons::pistol>(md, world, mixer,
                ↪ am); break;
            case 1: w = std::make_unique<combat::weapons::smg>(md, world, mixer, am);
                ↪ break;
            case 2: w = std::make_unique<combat::weapons::rifle>(md, world, mixer,
                ↪ am); break;
            case 3: w = std::make_unique<combat::weapons::shotgun>(md, world, mixer,
                ↪ am); break;
        }
    }
}
```

```

        case 4: w = std::make_unique<combat::weapons::sniper_rifle>(md, world,
            ↪ mixer, am); break;
        case 5: w = std::make_unique<combat::weapons::plasma_gun>(md, world,
            ↪ mixer, am); break;
        case 6: w = std::make_unique<combat::weapons::katana>(md, world, mixer,
            ↪ am); break;
        default: return nullptr;
    }

    return std::make_unique<weapon_pickup>(pos, z, tex, std::move(w),
        ↪ static_cast<int>(subtype), player, md, sub_id);
}

static std::unique_ptr<pickup> make_ammo_pickup(
    std::uint32_t subtype, math::vec2 pos, float z,
    entities::player& player, geometry::map_data& md,
    util::indexed_storage<geometry::subsector>::id_t sub_id)
{
    assets::texture_id tex = ammo_sprite_base +
    ↪ static_cast<assets::texture_id>(subtype);
    constexpr int default_ammo = 1;

    switch (subtype) {
        case 0: return
            ↪ std::make_unique<ammo_pickup<combat::weapons::pistol>>(pos, z, tex,
            ↪ default_ammo, player, md, sub_id);
        case 1: return std::make_unique<ammo_pickup<combat::weapons::smg>>(pos,
            ↪ z, tex, default_ammo, player, md, sub_id);
        case 2: return std::make_unique<ammo_pickup<combat::weapons::rifle>>(pos,
            ↪ z, tex, default_ammo, player, md, sub_id);
        case 3: return
            ↪ std::make_unique<ammo_pickup<combat::weapons::shotgun>>(pos, z, tex,
            ↪ default_ammo, player, md, sub_id);
        case 4: return
            ↪ std::make_unique<ammo_pickup<combat::weapons::sniper_rifle>>(pos, z,
            ↪ tex, default_ammo, player, md, sub_id);
        case 5: return
            ↪ std::make_unique<ammo_pickup<combat::weapons::plasma_gun>>(pos, z,
            ↪ tex, default_ammo, player, md, sub_id);
        default: return nullptr;
    }
}

std::unique_ptr<pickup> make_pickup(geometry::pickup_spawn const& ps,
    entities::player& player,
    geometry::map_data& md,
    ↪ util::indexed_storage<geometry::subsector>::id_t sub_id,
    engine::world& world,
    audio::audio_mixer& mixer,
    assets::asset_manager const& am)
{
    auto type = ps.type;
    auto subtype = ps.subtype;
    auto pos = ps.pos;
    auto z = ps.z;
    switch (type) {
        case 0: return std::make_unique<health_pickup>(pos, z, health_sprite,
            ↪ 25.0f, player, md, sub_id);
    }
}

```

```
case 1: return std::make_unique<armor_pickup>(pos, z, armor_sprite,  
    ↪ 25.0f, player, md, sub_id);  
case 2: return make_weapon_pickup(subtype, pos, z, player, md, sub_id,  
    ↪ world, mixer, am);  
case 3: return make_ammo_pickup(subtype, pos, z, player, md, sub_id);  
default: return nullptr;  
    }  
}  
}
```

world_objects/pickup.cpp

```
#include "pickup.h"
#include "entities/player.h"
#include "rendering/sprite.h"
#include "geometry/map-data.h"

namespace world_object {

pickup::pickup(math::vec2 position, float z, assets::texture_id tex,
               entities::player& pl, geometry::map_data& md,
               util::indexed_storage<geometry::subsector>::id_t sub_id,
               float radius, float scale)
    : renderable_entity(position, z, tex, scale),
      pickup_radius(radius), player_ref(pl), map_ref(md), subsector_id(sub_id) {}

bool pickup::is_consumed() const { return consumed; }

void pickup::update(float) {
    if (consumed) return;

    auto* spr = dynamic_cast<rendering::sprite*>(&player_ref);
    if (!spr) return;

    math::vec2 diff = (*spr)("pos"_f) - (*this)("pos"_f);
    if (diff.sqr_len() >= pickup_radius * pickup_radius) return;

    on_pickup(player_ref);
    map_ref("subsectors"_f)[subsector_id].remove_sprite(this);
    consumed = true;
}

}
```